

软件仓库管理

完整转录与课堂画面



画面 00:00:45

但设计上还是要骂

我好像还可以再苟活几个月

- 当然这是因为面对了非常开放的 intent-spec-impl 空间
- ◆ “做一个好用的界面”：什么叫好用，往往看到第一版才说得清。
 - ◆ 实现会反过来改变需求；设计过程中，题目本身也在变。

开放的问题，也能逐步变得可求解

- ◆ Simon 在 1973 年指出：结构良好与结构不良的问题之间，边界是流动的。(The Structure of Ill Structured Problems)
- ◆ 由此看设计：每一次反馈，都在把“我想要”变成更明确的约束；今天还要骂，并不意味着这会是永久性的能力边界。

南京大学·绿导师原谅你了

01:40:13 / 145 段转写 / 59 张课堂截图

正文来自夸克转写。少量明确术语修正已留痕；保留口语重复和原有语序。章节标题仅作导航，不替代原文。

时间说明：转写仅提供段落开始时间；截图标注实际画面时间。未逐句听校，其他识别错误可能仍存在。

原视频：<https://www.bilibili.com/video/BV1kybV6DE47/>

阅读导航

- 00:00:00 模型进步与为什么还要学软件工程
- 00:09:05 从项目目录到软件仓库
- 00:13:46 从 U 盘、压缩包到 Git
- 00:17:14 如何从成熟项目学习工程实践
- 00:24:40 Git 为什么成为基础设施
- 00:29:10 版本控制的由来：从备份到快照
- 00:35:01 Git 初始化与可审计提交
- 00:45:12 暂存区、提交粒度与 .gitignore
- 00:53:08 定位问题：blame、bisect 与自动检查
- 01:03:07 Git 的底层数据结构
- 01:08:30 用可视化理解提交、分支与合并
- 01:18:30 用 AI 构建可视化编程环境
- 01:26:30 最佳实践、Conventional Commits 与线性历史
- 01:33:25 总结：用原子变更管理 Agent 项目

阅读提示

每段正文前的时间来自转写文件。图片与文字按时间顺序编排，图片在段落间的位置仅用于阅读，不表示字幕具有逐句结束时间。原始 TXT、修订记录及全部准备截图均保留。

01 模型进步与为什么还要学软件工程

[00:00:00] 好的，在前几次课的时候我们讲了怎么用AI的一些案例，吧？就像我刚才给大家现场教学了一下，你们应该怎么把AI整合到你们生活的流程当中。因为你们在生活中会遇到无穷多这种不顺心，就是你你你用很多东西时候都会感到不顺心。但是AI作为一个非常强大的assistant，他可以帮你把不顺心的事情给挡掉。你只需要告诉他你的意图，真实的意图是什么就可以了。然后你甚至还可以蒸馏成skills，然后让这个过程变得更平顺。而且在这个过程当中，我们不断的看到这个新的模型的进展。

[00:00:37] GPT-6就真的发布了，大家用上了，然后我用了一下，反正还是有惊喜的。虽然说可能编程上的提升有限，他没有在编程上说的我那种我说我们现在有经验的软件工程师，还有比较好的品位，这品味可以指导AI做出更好的东西。那现在AI更多的时候是完成任务，好像GPT-6还没有学会这种非常好的品味，我经常还要骂他，所以我觉得我可能还能再苟活几个月。但是确实它在各方面的能力都提升了，而且做更复杂的任务的时候，它变得更快了，省很多token。我特意measure了一下，真的是是这样的，当然了这也是一个我觉得就要在这门课上讨论的问题。



画面 00:00:45

但设计上还是要骂

我好像还可以再苟活几个月

- 当然这是因为面对了非常开放的 intent-spec-impl 空间
- ◆ “做一个好用的界面”：什么叫好用，往往看到第一版才说得清。
 - ◆ 实现会反过来改变需求；设计过程中，题目本身也在变。

开放的问题，也能逐步变得可求解

- ◆ Simon 在 1973 年指出：结构良好与结构不良的问题之间，边界是流动的。(The Structure of Ill Structured Problems)
- ◆ 由此看设计：每一次反馈，都在把“我想要”变成更明确的约束：今天还要骂，并不意味着这会是永久的能力边界。

2/3



画面 00:01:00

但设计上还是要骂

我好像还可以再苟活几个月

- 当然这是因为面对了非常开放的 intent-spec-impl 空间
- ◆ “做一个好用的界面”：什么叫好用，往往看到第一版才说得清。
 - ◆ 实现会反过来改变需求；设计过程中，题目本身也在变。

开放的问题，也能逐步变得可求解

- ◆ Simon 在 1973 年指出：结构良好与结构不良的问题之间，边界是流动的。(The Structure of Ill Structured Problems)
- ◆ 由此看设计：每一次反馈，都在把“我想要”变成更明确的约束：今天还要骂，并不意味着这会是永久的能力边界。

2/3

[00:01:24] 就是因为为什么我什么时候我们会觉得这个GPT做的不好，还是因为我面对了一个非常开放的inten的空间。你脑袋中的比比如说同样的一个任务，如果我我我告诉你，就给我做一个国民级的应用。那么每一个人哪怕是顶级的产品经理，他脑袋里面想的都不一样，然后AI是没有办法和人脑去直接做这样的一个对齐的。所以他总一定会按照他自己的方式去漂移。

[00:01:52] 而且我发现AI写出来代码，人类越来越难读了。这个GPT-6我不知道是不是因为受他这个思维链变短的影响，它思维链变得非常紧凑，然后现在写的代码也非常紧凑。你可明显的从比如说GPT-5开

始到5.6，然后到6，然后他写的代码就写得越来越紧凑，然后你越来越不想看它，就人没有办法review veal那个代码。好像曾经好像我们会各种各样的prompt AI说你写出符合人类工程规范的代码。但好像现在在我们觉得我们也不我们再也不会看AI写的代码了，所以就随他去，他反正能把代码代码弄对然后我觉得GPT-6给我们最大的惊喜是它的空间感知和理解的能力。我一直认为OpenAI对这个AGI的理解是这样的首先AI得学会数学，因为数学是所有人类世界里面最难的那种语言，对吧？因为它我上第一次跟时候说，它有一个世界上所有人都认同的一个推理系统。

[00:02:56] 所以它他和我们说的软件工程里面的intent-spec-implementation的gap是反过来走的。数学是有全球性的共识的，你你你一个定理你你能证明出来，那么全球的人都会认为这个定理的证明是对或者不对，所以它是一个非常好的能grounding的东西。而且数学的逻辑推理是可以可以解释我们人类生活里面的各种行为的。就比如说你会觉得好今天中午吃什么？你们马上就会面临这个问题了，去挤食堂。这个问问题，你们可能会觉得这是一个人类世界里面的问题，必须是你的灵魂来给出答案的。但其实不完全，因为你你的大脑里面其实不停的在投票，对吧？比如说我就会选择在那个艺附楼订饭，因为对我来说这是最快的这个选项。因为我下午还要开会，然后如果如果我时间能来得及的话，我中午可以小睡一会儿。Ok所以你看我其实是在各种因素之间投票。

[00:03:53] 然后在这个因素之间，比如说什么事情，a impb对吧？如果我做了A就会做B这其实是一个数学。我可以到你们学在离散数学里面学那个命题逻辑里面的各种各样的东西。所以OpenAI应该是说我先把数学学会好，现在数学它已经学会了，然后学会了以后他再去强化学习别的能力。



画面 00:04:00

“理应当做到”的事就是能做到

我对 OpenAI AGI Roadmap 的理解：一切对齐到数学逻辑推理

- 直接杀死了比赛 (具身赛道的人应该在瑟瑟发抖)



做了小半年的工作直接了

Baseline 应该是不可能比得过了。心疼 PhD Student 一分钟

别上班就躺平吧

© 地点 泰康之家养老社区 课程 1.8h

[00:04:13] 我一直觉得比如说空间感知，他他不是这种绝对学不会的。我我就这样想，就比如说给你一个鲁班锁，那个很复杂结构，然后我就问你你按照什么样的方向可以把第一块东西给抽出来，对吧？然后你们人脑就会开始你也做过那种智力题，我觉得你们从小就会做。说给你一个立方体或者一个什么奇怪形状的投影，就有些那种bench mark里面就是这样的。然后问问这个哪一块和它是一样的，或者给你一个骰子对吧？六面的骰子问问哪一个展开图对应了你看到那个骰子，然后你就在你的脑袋里面做这种空间想象。然后这种空间想象你只要从简单的任务开始给他强化学习，肯定是能做到的。

[00:05:00] 然后马上我我当时看到这个例子说那时候GPT-6还没有发，然后我就心疼我的这个d student。因为我们已经做了小半年的3D打印的工作了。然后我们可以在GPT-5.6的空间感知能力很差的情况下，后可以通过直接对话，可以生成一个很像样，而且各方面都能符合规范的3D打印模型。

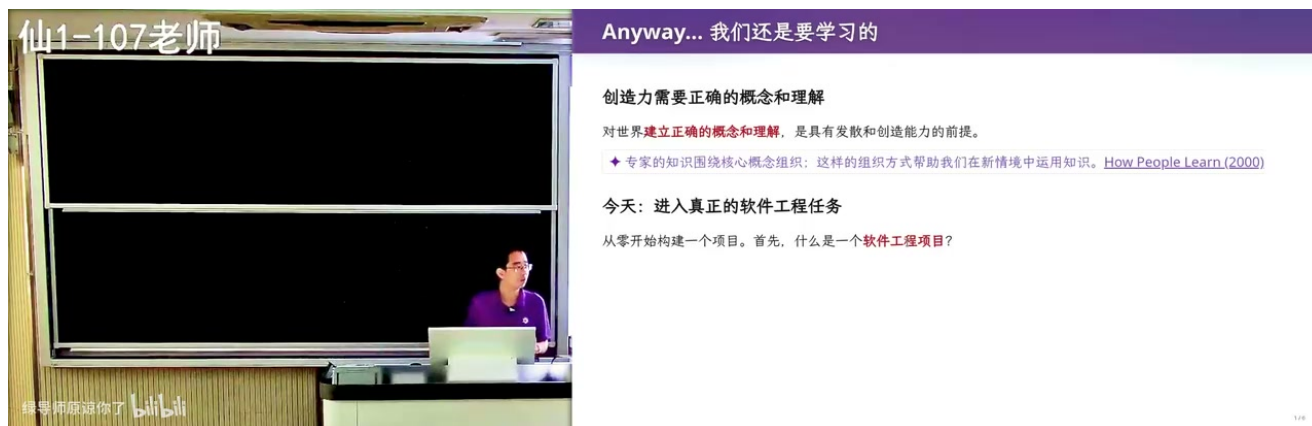
[00:05:23] 我们还是有些比较pler的application的，比如说我们可以做一个那种小的，我不知道你们有没有玩过那种光山玩具，就是那种光纤尺。你左从左边看和右边看是两个不同的图案。然后比如说你可以说我要做这个，然后我也可以一个很长的搜索数，然后帮你做一个光圈尺。

[00:05:42] 然后结果昨天这GPT-6 one shot直接出，直接搞定。然后我们现在还在想想怎么样解决解决这个问题。然后同样的是因为他有了空间的感知能力，又有数学的能力以后，他就他在聚身赛道就很明显。是可以可以做的很好的。所以第二天你又看到很多人在小红书上发他直接用GPT-6去控制一个机械臂，去做这种抓取或者叠衣服这样的复杂的动作。而注意个GPT-6是一个语言模型，它是？虽然它有视觉的能力，他就是一帧一帧的问你下面要干什么，还没有解锁这个提示词的engineering，然后他就已经做的相当的好的，已经达到了一个state of art的能力范围。所以模型一定会进步的。

[00:06:33] 这个例子告诉大家，也许在我上课上到某一次的时候，我突然就觉以说这个课不用上了。这样也好，然后大家就可以去做产品了。Anyway到今天我觉得我们还是当然需要学习一些软件工程的知识的。因为AI我刚才说AI解锁的是你的执行力。我我举我举这样个例子，因为我正好被人问到，我们在AI的时代为什么还要不要学数据库，学操作系统，学这些学学这些东西。



画面 00:06:43



画面 00:07:00

[00:07:08] 然后我的答案其实也不是特别的确定。因为我打过这样的一个比方，我们回到一百多年以前，汽车刚发明的時候，那你你汽车开个一两公里就会出故障。所以你的每一个司机，每一个driver，你都必须对汽车的工作原理了解的非常清楚。因为你的汽车一旦出故障，你就得知道它是怎么样工作的对吧？那在那个时代，我们把人回到那个时代，这就是理所应当的事儿，对吧？

[00:07:35] 你你你想要你要开车，你就需要掌握汽车的基本原理的知识。你要学computer science，你要去软件工程的工厂，那你就去学操作系统，学数据库，把这些基础知识做好。但你想想今天我们怎么开车，对吧？今天我们在地下车库直接说帮我开到什么地方，然后自动驾驶汽车就出发了。然后这你是不可能，或者你作为一个人已经不理解整个系统站上面是怎么工作的了，对吧？

[00:08:04] 因为最终它的动力还是传递到一个机械装置，和100多年前汽车还是有一点相似的对吧？但是在这个过程中，他经历了所有东西你都不需要知道，你就可以把汽车开起来。换句话说，人工智能你也可以想象成是它是一个类似于汽车这样的东西。当然它比汽车更强，它替代的不仅是人的腿部的向前走的能力，它替代人的这里以后，我们我们是不是还要学，对吧？

[00:08:31] 当然我今天答案肯定是要学，因为就你们学了以后，你可以更好的解锁这个AI的创造力。但是以后应该怎么学，我现在还暂时是暂时是不知道的。你们也许如果你们有答案的话，也可以欢迎和我来来探讨。当然这是我的个人的观点，你们至少在现在的阶段学习还是有好处的。你理解了正确的概念，可以帮助你从别的地方借鉴一些知识，来创造更好的东西。然后我今天就会进入真正的软件工程，对吧？

02 从项目目录到软件仓库

[00:09:05] 我们说生成式软件工程。那你如果要开始做软件工程，开始Vibe Coding，你干的第一件事情是什么？你不是从网上或者说比如说做作业的时候，那你一定是从课程网站上或者老师的要求的什么地方下载一个东西，然后从那里开始工作。

[00:09:25] 如果你今天就真的是想要做一个项目，那你做的第一件事一定是先建一个目录，对吧？建一个目录，然后给它起一个名字，起一个中文的名字，叫我的最酷的game对吧？然后你就开始Vibe Coding。那什么是一个软件工程的项目对吧？你创建了以后，你面对一个空的目录的时候，你们还记得你们是怎么学会应该做什么呢？对吧？

[00:09:53] 你们可能会看到老师在上课的时候打开了一个工程项目，然后里面有一些文件，你就不经意之间记住了一些这个项目的东西。然后你可能会会老师会向你推荐，或者同学会推荐，什么东西发布了，你可以去看看。然后你得到了一个什么GitHub或者g体的链接，然后你打开以后，那是一个网页，网页还是非常人类friendly的。你看到那个网页上面有一些文件，你就知道我的一个目录里面会要有一个文件，对吧？所以当我们说我们要开始一个软件工程项目的时候，你就会建一个这样的目录，它里面有一堆文件，然后这堆文件共同构成了一个可以交付的东西，对吧？这个东西最后会变成你的产品。

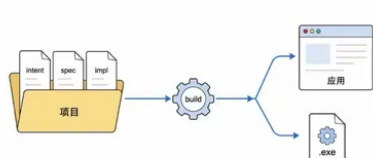


画面 00:10:00

什么是一个软件工程项目？

一个目录，一组文件，一个可交付的东西

- 目录里的文件，混合了 intent、spec、impl、.....
- 经过 build / 运行，可以得到 npm run dev 的本地窗口，也可以是一个 .exe。



[00:10:40] 这个目录它可能是直接在你本地运行的。在你神奇的跟跟这个agent说，请你帮我运行的时候，那个窗口就弹出来，或者网页就弹出来，对吧？一个著名的笑话说GPT-6太牛逼了，我又做了多少个killer级的产品，他们是HEVD local host下的这个5137对吧？然后发一个ttr对呃，所以你还是需要支持的对吧？你在你本机可以打开它，但是那世界上的其他人打不开它。因为如果你不知道localhost是你的

本，但本机的话ok当然你可以通过把你的intent speci说的更清楚一点，对吧？如果你在你的intent里面，比如说在那个文档里面，就说明我的这样的一个东西要被全世界人看到。如果你不说这个GPT可能会把你当成一个开发者，那开发者最正常的一定是在你本地先调试，对吧？

[00:11:34] 先起一个本地的development server，然后你可以看到它的结果。然直到你觉得ready了，你才会把这个东西送到原端去。但这都是知识对吧？

[00:11:43] 你们在计算机，你们是science的学生，上各种各样课的时候，反复的学习这些知识。然后这些知识最后ok变成这个对，变成这个。好，它里面有文件，最后会得到一个可交付的东西。然后你们会从最开始学编程，可能是你在which里面一键编译和运行。然后到后面你们学的时候会知道你的build的目录里面。

[00:12:12] 所有的东西都是由某些命令行的工具创建出来的对吧？你们会开始在大2的时候用一些像GCC这样的命令行工具，会把一个hello点C变成hello的一个binary，这就构成了一个软件工程的项目。所以你们会听到这样的一个标准的定义，叫什么叫软件？这是一个很好的问题对吧？以前老先生喜欢问我们什么什么是软件？软件是一个如此基础的东西，到处无处不在的软件，对吧？

[00:12:41] 你想没有软件，你们今天看到这个是黑的。你不光你不看到我这个，你就连这个台子也启动不了。我这个升降，就连我的这个升降桌，这个教室还是不错的。有一个可以升降的升降桌。你看如果没有软件，我这个升降桌都是升不起来降不起来的。这个升降桌这边还有一个高度的显示，现在是106厘米。那么软件是到底是什么呢？吧？

[00:13:08] 有个标准的定义叫什么代码文档等一切和软件相关的东西的集合，对吧？好像有一个这样的definition。但其实我我我我一直说软件是人类世界的需求。在信息世界有一个投影，然后这个投影今天看到是一个目录，然后目录里面有各种各样的东西，ok that's all对吧？你知道目录里面应该有什么，比如说一个hello点C然后你可以编译出一个hello，你就已经在做软件了。当然这个可能不符合现代软件工程的best practice，对吧？但古早的时期，我觉得就甚至是十年以前，大家真的是这样的。

03 从 U 盘、压缩包到 Git

[00:13:46] 我给大家讲一个我的第一个PhD student的故事。他应该是2012年入学的，进入南大的。所以第二我第一个博士他他大一的时候，他发现了一个非常非常好的学习的方式。因为他经常会去机房，他可能他自己的笔记本电脑还是屏幕比较小，也不是很舒服。不像现在我相信大家应该是都会买一块屏幕应该不贵对吧？不是很贵。甚至淘一个二手的屏幕只要小几百块钱，然后你就可以在宿舍获得非常好的编程变成体验。



画面 00:14:00

古早时期: That's All

一个优盘，装着全部代码

- 我的第一个 PhD Student 的故事 (约 2012): 大一把代码保存在优盘上; 优盘坏了, 代码没了。
- 交作业: 打包发邮件; 现在似乎变成了网页上传。



优盘坏了, 代码没了

打包 -> 发邮件

3/8

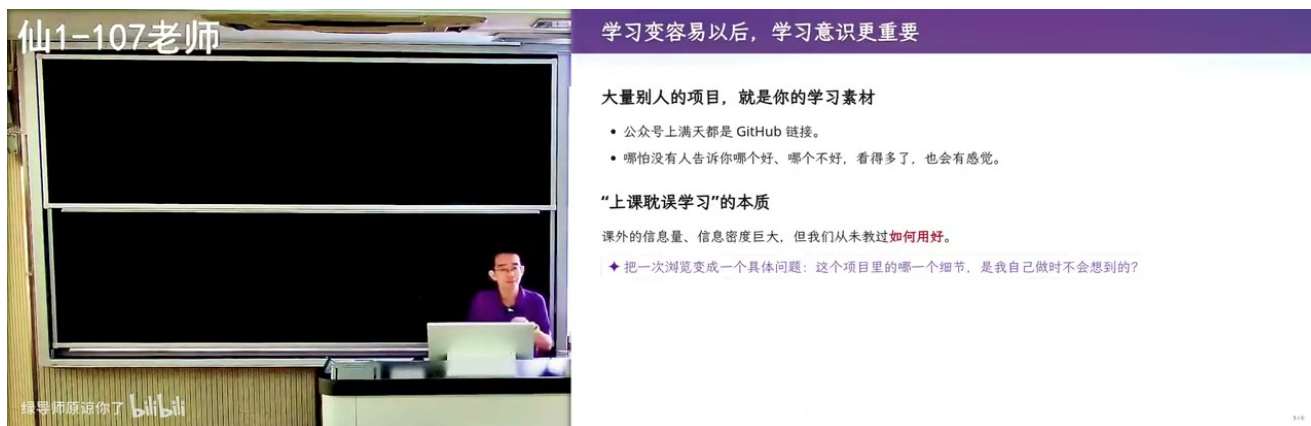
[00:14:19] 那个时候好像我们的标准答案是去机房，屏幕好像没有那么便宜。那去机房的话他就会要来回拷贝代码，比较麻烦。所以他想出了一个妙招，大一的时候把代码保存到U盘上。你们多多少少可能也干过这样的事儿，或者在中学的时候，上电脑课的时候，你做的作品你会保存在U盘上。然后结果到达大二的时候，这个U盘坏了。然后大U盘坏了以后，代码全部都都没都没有了。作业不见了，然后他珍藏的那些？启蒙的时候的作业不见了，有一点伤心，但是好像也没有太大的关系。

[00:14:54] 对我们我们很长时间都是这样管理代码的。比如说我记得我上学的时候，交作业的方式是打一个压缩包发邮件。然后现在好像你们会在一个网页上面网页上面上传，这都是管理项目的方式。但是软件工程在我们当我们谈软件工程的时候，今天是有有标准答案的。就是你应该去用Git是一个除了能够保存一个目录，就是项目是一个目录。目录里面有hello点C1，有这个hello的binary，还有文档等等。它还能保留比较和恢复历史的版本。这是一个很重要的软件工程的特性，我后面会详细的展开来讲。

[00:15:39] 然后有了g以后，其实很多事情都变得很容易。比如说我再也不需要发邮件了。我真的很难想象就是我们过去今天是agent的时代，你们我有了这个十倍100倍的生产力，我只要我只要让他去帮我写一个投诉compla我，他就他就我就什么也不用管，对吧？我就我比如说如果我我今天是用我的codex，我就真的选择信任他，我就我真的就不管了，这事就过去了，我看都不会，甚至我看都不会看我让他把结束以后把这个浏览器给关掉。但过去的时候真的我们还在用邮件管理项目。

[00:16:16] 然后有了g以后，我们发现效率高了很多。有我们可以用git clone去从一个远端获取一个作业，对吧？然后如果作业更新了，我我会我我可以在远端更新，我就不需要给每一个同学都发一个什么所，我只需要在上课的时候告诉同学们说，我的作业更新了，你们用pool去下载。然后以及我我的比如说我上课的时候，我有一个mix submit，你会它会自动在用脚本把包打打好，打好以后上传到我的课程网站，非常的方便。这有点像一个现代的D的流程，应专门会有一次课课来课来讲。然后这就引出了一个我觉得这还是我每几乎我每节课都会要讲的讲讲的事情。就是当今天我们学习变容易了以后，这门课上讲的知识都不重要，重要的其实是你们的学习意识。

04 如何从成熟项目学习工程实践



仙1-107老师

学习变容易以后，学习意识更重要

大量别人的项目，就是你的学习素材

- 公众号上满天都是 GitHub 链接。
- 哪怕没有人告诉你哪个好、哪个不好，看得多了，也会有感觉。

“上课耽误学习”的本质

课外的信息量、信息密度巨大，但我们从未教过**如何用好**。

✦ 把一次浏览变成一个具体问题：这个项目里的哪一个细节，是我自己做时不会想到的？

画面 00:17:00

[00:17:14] 比如说你们回想一下，你们都看过很多g的项目，对吧？看肯定的。因为你哪怕是打开课程作业，你就会看到它有一个Git的GitHub的链接。那么你们有没有留意过不同的项目它们之间细微的区别？比如说有的项目有这个文件，有的项目没有那个文件。然后有的项目里面文档是这样写的，有的项目文档是那样写的。你在这种不停的有这种信号进入的时候，其实你就能判断出我们应该怎么用Git对吧？

[00:17:45] 我不知道大家是不是总有一种感觉是老师老老师说这个是不是跟中学的时候有关系，就是因为老师说的事情实在是太多了。在中学的时候老老师你必需要听老师的话，你你你才能把所有的事情做完。如果你不听老师话，你就不可能把事情做完了。但到了现在，比如说你去GitHub呃学习的时候，如果你还带着个心态，老师说好你去把作业拉下来，那你就就是你你就像AI对吧？就有点像今天干活的AI你被强化学习成了一个当老师说让你做这件事的时候，你只做这件事绝对不会多碰一点点的。因为这是最省时间的对吧？AI也被调成这样了。但如果你花一点时愿意，其实我就劝大家愿意多花一点点时间去点进去看一下里面的各种东西。

[00:18:38] 然后你你你可能突然间就有个灵光一现，哎呦，这个文档写的很好，因为你马上觉如今天你看到那个满世界的Git GitHub上面项目全部都是巨长无比的read me，然这里面有各种uniic的字符组成的什么目录结构框图对吧？就这种AI sla感觉你马上就有了，你们有没有留意过这些细节？然后在这些你看过这么多AI slo的时候，突然你发现了一个什么老项目，哪年几十几年前开始维护的老项目。点进去那个文档清新舒服，read me很短，但是read me看完以后你想看的东西三个链接你就走过去了，你不需要看那么长的AI AI，你就学会了文档应该这样写对吧？

[00:19:24] 我因为我我觉得你们我不知道你们有没有有机会去，比如说去对比不同的城市的路牌。然后我就觉得是很有意思的，就为为什么为什么我们有时候觉得中国是一个developing country，对吧？是一个发展中国家，意味着我们的个提升的空间很大。我觉得我有一次，已经好多好多年以前了，我特别Git impressed。

[00:19:50] 我在香港的时候我发现我要找路。然后我在找路的时候，我发现每当我有不确定性的时候，比如说我前面有三个口的时候，正前方一定有一个路牌告诉我三个口是什么。从来没有过，就是它有一个非常复杂的地下系统。然后有的时候你回到南京的时候，你发现你有的时候经常会要在地铁站里找这个找我下一步对吧？我不知道了以后要到哪一个口的时候，说你需要环顾四周去寻找，甚至要移动比较远的距离找到一个地图地图再去看。然后你其实这是一个学习的过程，我觉得很很好。如果有一个对大家来说有一个范例比你做的好，然后你就学到了。我其实在这个对比当中，我就学到一个lesson。

[00:20:36] 比如说如果我要做一个UI设计的话，我的这个UI就应该是任何人我会要要站在人的角度，人站在这个页面上的时候，他会想什么？不同的人他要的ABC它会去向不同的方向。我希望用最少的方式把abc呈现全部呈现出来。所以你看这就一个read me你们都会用AI生成sloread me。但当你看过几个好的以后，你就会写me了。其实这我告诉大家，因为你有AI以后，你马上就可以问他，这个me为什么比那个为什么为什么我看到这个项目的me没有a AI slo的感觉他就会告诉你，所以这也是我理解的，就为什么很多同学们怨气很大，觉得上课耽误学习。因为现在课外的信息量和信息密度已经远远超过任何一个课堂可以做的事情了，对吧？

[00:21:29] 你当你打开一个成熟的GitHub的项目的时候，它是这个工程师十几年的心血。甚至在某一个时间点，十年以前这个项目崩溃过一次。他做了一次大版本的重构。他发现以前的那个practice不够好，你才呈现到你面前的是一个现在好的东西。

[00:21:50] 但是我们传统的教学方法没有教过大家怎么样把这些海量的信息密度巨大的东西用好。然后大家也没有准备好去拥抱这样的知识，对吧？可能呢可能为什么我现在在做老师？就是因为我有这种惊人的注意力。然后大家其实要训练的也是那种惊人的注意力。你可以在看到一个东西的时候，快速的有种什么感觉呢？教教大家怎么学习。

[00:22:18] 如果你有任何一个问题，你看到东西之前，你的大我的大脑有一种可能有一种能力，就会先把这个东西我看到的所有东西都忽略掉。我会想如果我要自己来解决这个问题，我会怎么做？然后我会立即比较我看到这个东西和我脑袋里所想的的东西谁更好。然后如果我我我做的更好，就认为这是一个slo然。后如果我发现他做的比我好，我就很惊喜说他为什么做的比我好？然后如果你养成了这个习惯以后，这是一个非常恐怖的习惯，对吧？因为你在任何时候都在疯狂的从外面接收信息，然后你能留意到生活里面各种小细节的时候，你就不停的在在这个提升。



仙1-107老师

画面 00:23:00

留意：让每一次阅读都成为学习

Virtually Implement：先在脑中做一遍

- 看到问题，先问自己：我会怎么想？如何 approach？
- 再看作者的做法：
- 如果一致——很好。
- 如果不一致——就是一个学习的机会。

对细节保持注意力

- 读别人的东西，甚至 AI 生成的东西，都要有“感知细节”的意识。
- 哪怕只看到一段话，也可以预测下一段是什么。
- **留意**使你持续学习、水平持续上升。

◆ 对照时再追问一句：“这个差异来自什么约束或假设？”“学习研究中的“自我解释”，也关注把解题步骤与背后的原理联系起来。[Chi et al. \(1989\)](#)”

[00:23:00] 然后也有其实很早的时候大家就有推荐，如果你们要读一个research paper，读一个研究论文应该怎么读？我早先的时候展示AI是怎样被这个注意力机制带到作者这个逻辑陷阱里面去了。他说有一个非常好的学习方法，这个学习方法其实也适用于大家学任何课程，包括你们的高等数学，包括你们的计算机系统基础。

[00:23:24] 叫virtual implement，就是你现在你你有一个具体的问题或者具体的需求，你先在脑袋里面把这个问题自己做一遍，你会得到一个什么样的结果呢？如果你的方法和作者的方法是一致的那很好说明你已经学会了，完全掌握这个知识了然如果你不一致，还有两种情况，一种是确实那个成熟的好的教科书上的方案比你想要那个方案要好，要精巧，要简洁，你就会想为什么为什么他能想到，他就变成你知识的一部分。还有一种情况，那就真的是惊喜中的惊喜，你的方法比他还要好，对吧？可能你1万次

才会有一次这样的体验，但是但这个体验给你震反馈也很大，对吧？你突然会想我能不能去做一个什么什么好玩的东西。对所以我觉得在AI时代，你们更可以对细节保持注意力。比如说git你看到各种各样的g的，你读别人东西，甚至AI生成的东西都有感知细节的个方式。对，这是我开一个开一个头，然后我们就可以讲关于Git的一些具体的细节。

05 Git 为什么成为基础设施

[00:24:40] 首先为什么要Git对吧？大家有没有想想过，当然肯定有GitHub以后分发代码变得前所未有的方便。这件事情可能创他们创立gi他的时候就想到了，但他们没有意识到，这竟然成为了比如说AI时代的一个底层的基础设施，对吧？NPM你的所有的各种M的配置全部都在都都在gi上。我就在上周末的时候，我参加了推免的面试。你们应该也在一年或者两年以后会来到这个场合去去面试。



画面 00:25:00

Git：听说过，还是用过？

上周的保研面试

- 解释什么是 Git：多多少少都能说两句。
- git rebase 是什么：0/10。
- git merge 冲突以后，会观察到什么？
 - 竟然没有人说出：文本内容冲突时，双方内容和冲突标记会直接写入工作区文件；git status 显示 both modified。

第一没用过，第二没有留意

如果平时刷各种材料，多多少少会刷到：真正用过，也要留意眼前发生的事。

✦ 观察入口：打开冲突文件，看看 Git 留下了什么。

[00:25:16] 然后我就问大家，什么是Git什么是Git？那什么是Git？大家多多少少都能说两句对吧？我从Git上面可以下载分发代码，然后有的同学可以讲到Git管理的一些这个目录项目目录的快照。然后好挺好的，我当然我觉得也没有必要把所有人都挂掉比对吧？你能讲一些就可以，就取决于你能讲多少，来给你一个评估。然后gbase是零of 10，这个也意料之内。

[00:25:48] Ok但我比较意料之外的是，大家我问大家，如果Git merge发生冲突以后会观察到什么？你们有观察到过吗？是如果你比如说你的作业老师发布了一个更新，但你本地已经把这些东西改得面目全非了。那这个时候你还是会说让agent说来吧，然后agent会自动就*****，就给你修好了，反正说给你说好了。然后我现在也是的，反正位置出错了帮我修。

[00:26:26] 反正最后他说好测试都通过了，然后我看一眼这个东西看起来ok，这个时候惊人的注意力就有用了。如果你愿意在冲突的时候停下来，尤其是大家作为学生的时候，总是想我再学一点，反正这一秒钟我学校也也没有没有坏处。你会看到一个非常神奇的？有好多等号，有好多箭头。然后你就是在你们之冲突以后，和不是立即这个冲突就就解就就被就被resolve了，而是你这个责任就到了你手传在成经的时候是到了每一个开发者手上。如果开发者发现了冲突，那开发者有义务要把这个冲突给解决掉。那他要解决掉，他就需要能看到这个冲突，对吧？他不能说。

[00:27:11] 如果Git是设计成你你站在Git设计者的角度，如果两边这个呃比如说我现在有一个文件，文件原来是一一，然后有一个人把第二行改成了2，有一个人把第二行改成了3，那这个到底是二还是三呢？我不能就Git说对不起冲突了，你我不能合并了，然后躺在那儿，那对人来说这个负担是非常大的对吧？

那我到底这行应该是以二放进来，还是应该以三放进来呢？那我能不能需求就是你看所有的1st principle都是连起来的。

[00:27:46] 我刚才无意中举的那个地铁标识的例子，Git也在用这个原则。就是如果你现在发生了一件事儿，那最好就是在这个地方把它所有要的信息全部都放进来。所以你你你会回想到，你看到了如果你以文本的方式来看，它有一个等于号，有好多间括号。然后有些地方有这种incoming的这种hair的这种标记，然后至少冲突的两部分代码都同时显示在这里。后如果你们用像ID的话，你还可以看到上面会多出一些按钮？什么except incoming change，然后你就可以在这个ID里面自己re自己决定谁进谁出。完了以后你就可以再创建comm把，它把这个给resolve，你只需要一点注意力，其实你只要有一两次，你就会有一个影响力对吧？你至少至少至少你能说出两边这个冲突的代码都会显示在一起，对吧？

[00:28:44] 然后比较惊吃惊的是这个大家都答的不好，那就肯定是第一个没有用过，大家没有开发过。当然我觉得就算用agent的开发，你也很容易遇到这种conflict。如果你让agent在在不同分支开发，第二还是没有留意。所以比较重要的是第一个去多用。第二个可以试图去拥有更好的注意力。

06 版本控制的由来：从备份到快照

[00:29:10] 好，那我们就来讲讲g是怎么来的。在那肯定我们在U盘上面放一个目录的方式是不行的对吧？每次打一个包，但据我的观察，我在上学的时候，同学们确实是这样。当然我上学的时候可能比较早了，这个g没有Git popular，作业V一作业V二作业，最终把每一次复制一个目录没问题，这个是一个非常直觉的体验。

[00:29:41] 这个方式可以找回旧的文件，对吧？因为你你你万一有一天手滑了或者不小心改错了，尤其你们在新手阶段，不小心有一个误操作是非常正常的。你想要回去上一个版本看一看到底我上一个版本是怎么写的。这不就是一个需求，然后就从这样的一个需求，你自己就能发明一个像g这样的东西。因为本质上所谓的版本管理？这我们说什么是项目呢？项目就是一堆东西？你的代码、文档、intent-spec-implementation各种东西，测试用例全部都搅在一起没问题。

[00:30:21] 一个目录，然后你会想如果我们能够把目录的版本管理起来，如果我不是不再是叫作业一、作业二、作业三，我只有一个作业一个作业的目录。但是我可以提供一个命令行工具说我能不能看一看list作业的版本是什么？我能不能把当前的版本变成另外一个版本，回到过去的一个版本。你不就发明了一个，虽然你还没有使用任何的什么分支，什么合并、冲突这些概念，但是你不就已经发明了一个简化版的git。

[00:31:03] 然后在你用着的时候，你自然而然的会去发明那些，对吧？比如说你要跟你的室友同步的，你就会出现他你们俩一起改同一个文件的情况。你就坐下来想，一起改了同一个文件应该怎么办呢？你就可以发明发明Git。当然我觉得这个人类所以这体现了人类的局限性。

[00:31:27] 在在Git之前，我们还用过你们可能听都没有听过，叫SVN的check-in/check-out这个。然后SVN和git它的设计就不一样。SVN是这样的，SVN说我们存储的是项目的变化。假如你有一个最初的版本，每次你这个文件改了一行，我就存一个patch再改一行，存一个patch好像没有问题。这是一个非常高效的实现，对吧？是一个非常高效的实现。但是这个高效随着项目的增长把大家拖回去了，拖入了泥潭。

[00:32:08] Git的这个模型要干净的多。就是大家1st principle我想要先回到过去的某一个版本，马上一秒钟就给我回去。而你想如果你每一次存的都是一个delta的话，你想回到一个早期的版本，你就得一个一

个往回退，对吧？当然你可以用一些缓存之类的方法来优化它。但这已就我觉得有种好像确实是走错走错路了。

[00:32:39] 所以Git基本上它就是这样来的，它的作者是Linux的作者，Linux一个非常简单干净的模型。他们在1991到2002年的时候，如果想要维护Linux就是用的作业V1，作业V2的方式。然后在邮件列表上面发补丁，直接发邮件，相当于是我改了一个A点C然后我说我有一个A点C新版，然后我就在往邮件列表一发，然后会有一个每天的会看到邮件列表一把它放到它的核心仓库里，然后再去release这个版本。然后直到后面他们用一些其他的工具keeper这样，然后直到release终于受不了了，他觉得所以我们就应该要一个绝对简单，但是又非常好用，概念正确的版本控制工具。所以05年的时候做了做了，所以它比我们想象的年轻一点，对吧？



画面 00:33:00

Git 从哪里来？

一个大项目，逼出了自己的工具



Git 简史 - Pro Git

[00:33:39] 2005年你有兴趣的同学可以去看这个pr大家直接按agent看就可以了，你不需要再去读了，我读书的时候我们都是看过这些文档的。对，所以that's it我们需要项目级的时间回溯，我们都会犯错误对吧？完错。



画面 00:34:00

我们要的是项目级“回溯时间”

人和 Agent 都会犯错误

- 设计会被改掉，对的代码会被改错.....
- 哪怕手滑把文件删掉了，只要已保存在 Git 历史中，就能捞回来。

Git 管理的是项目的快照

- 一次提交，记下纳入版本管理的文件当时的内容，以及它们在目录里的位置。
- 为什么要以项目为单位？代码和配置一起改过，只恢复旧代码、留下新配置，可能拼出一个从未正常运行过的版本。
- “存过盘”不等于“存进历史”：希望将来回到状态，需要提交保存。

快照与提交 - Pro Git

[00:34:00] 所以我们只要能去Git的历史里面把它捞回来就可以就就可以了。所以你们可以试着用这种方式去？从已有的经验你知道什么，我想要什么，然后可以推出下一个概念，它应该是什么样的。然后你发现，这就是数学的推理。我的就我上课的时候，我这样讲下来的时候，我每一句话都是和之前是有逻辑关联的。然后这个逻辑关联多多少少都有一些数学的联系，所以欧认为他要把数学先训好，训好以后他所有东西他问题他可能都能解决，然后这件事情可能就真的要能要能要能成真。我自己是觉得品味上可能品味可能也是也是能解决的。好，那我们还是讲一讲什么是Git，讲一讲什么是Git no还是行，我还是建一个空的terminal，建个terminal。

07 Git 初始化与可审计提交

[00:35:01] 大家也都知道，你今天如果要开启一个项目的话，那就git init对吧？It好，当然也不重要，这个它会告诉你一些奇怪的信息。那这个时候你就有两种选择对吧？第一种是让agent来帮你做，帮你解决，帮你回答你不懂的问题。还有一种就是把它忽略掉，那显然是高效的学习方法是更好的。然后你得到了一个空的，就这么简单以后你就得到了一个得到了一个目录，一个.git 目录。

[00:35:44] 当然.git 目录默认是不显示的，所以你你看不到它，这就是全部了当你做完Git以后，你的项目就变成了一个版本管理的被版本被Git版本管理的仓库。然后你可以从远处下载代码，可以把代码推到本地，然后你也可以最最基础的命令，比如说我有个hello点C？随便，你们喜欢写C，好，如果你有一个好作点C你就可以Git add，比如说好多点C对吧？然后就可以看到用status可以看到，我现在的这个工作区里面有一个hello点C，对，这里还有一些些提示。



仙1-107老师

绿导师原班你了 bilibili

从空白开始

Tutorial 告诉你的基本模型

git init 会得到一个空的 repository (repo)。可以使用一些不知道什么意思的命令，和 GitHub、GitLab、.....建立关联。

- git clone: 从远处下载代码
- git add: 可以“添加”到 Staging Area (Index)
- git commit: 可以“提交”一个快照

就可以开工了？

- ✦ git add hello.c 之后，又改了 hello.c，下一次 commit 会提交哪个版本？
- ✦ 提交的是 **add 时暂存的内容**。会运行一个命令，和知道它改变了什么，是两回事。 [Git 官方说明](#)

画面 00:36:00

[00:36:32] 然后这个很重要的是创建项目的快照，使得你可以在任意时间之间穿梭。所以我们创建快照的？我可以创建一个有消息的快照，这是这很重要，这体现了软件工程的智慧。你看之前我们用复制粘贴的方式命名为目录一，对，项目版本V1，项目版本V2，项目版本V3对吧？作业V1，作业V2，作业V3。然后k的世界里面不是这样的，你们我觉得我我真的我看到很多同学在用git的时候，把Git当成是作业V1、作业V2、作业V3来用的。



仙1-107老师

绿导师原班你了 bilibili

```
$ eza --git --group-directories-first --icons=auto
$ vi hello.c
$ eza --git --group-directories-first --icons=auto
@ hello.c
$ git add hello.c
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   hello.c

$
```

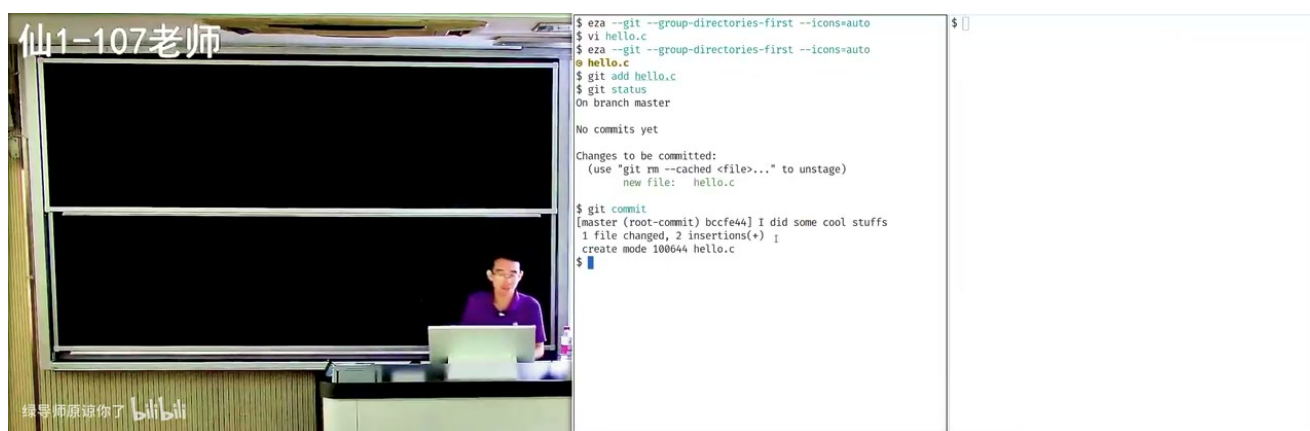
画面 00:37:00

[00:37:16] 但这个不是Git推荐的practice，Git推荐的practice是就或者说在长期的软件工程实现实践的经验当中，你每一次当对这个项目做了任何的改动以后，比如说改了一个文件，然后这个文件比如说修了一个bug，或者给他增加了一个功能，或者调整了某一个脚本，或者调整了某一个文档，你都应该立即做

一次git commit，对吧？都git commit。这怎么是nano？然后你可以有一个文本编辑器，说说我这次做了什么对吧？问变量好，我可以保存好，那这个提交就成功了。然后这些都你们都可以在现有的项目里面看到对吧？他们是怎么样协作的？有些Git提交可能比较大，他们放了一个feature，有些就比较小，它就这非常少的少的东西。所以当你面对这样的一个新鲜的的工具的时候，你去理解它的best practice还是需要花时间的。

[00:38:29] 所以我还是觉得你可能不能把所有的事情都交给A，你多多少少还会知道我们为什么要做件事，为什么？什么时候应该做git add什么时候应该做git commit？Git commit的message应该怎么写？各种各样的这样的知识。

[00:38:51] 所以你真正需要的是在任何时候学习。比如说碰到这个新东西的时候，追问一下你刚才做的是不是符合best practice对吧？比如说我刚才做了一个git commit，这个符合best practice吗？对吧？就问你加上逗号输入法，真的就是一秒钟的时间，对吧？

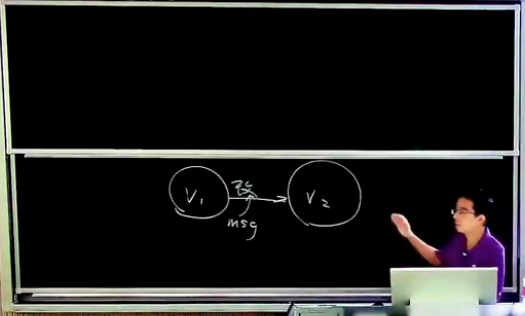


画面 00:39:00

[00:39:22] 但是他会给你很多解释，对他说这个不太符合best practice，主要问题在提交信息，其次代码本身也有问题，提交信息的可读性有问题，对吧？我故意做了一个esome steps，完全看不出做了什么。所以Git首先就它本质上，因为本质上虽然它存算存储的是快版本的快照，但是我们的项目本质上还是从一个版本V1，然后经过一次修改变到V2的修改。Git鼓励的是你需要在message里面描述出你改了什么。相当于你如果一直看看看好你就知道这个项目是怎么发展，一直走到现在的对吧？修了多少个bug，加了什么需求，经历了什么样的重构。你可以在message里面看到这个项目的历史的轨迹，这是他鼓励的。然后这这也是一个好的软件工程的practice，使得别人能够更容易的理解你的项目。

[00:40:31] 所以如果你的每一个提交都是I some cool stuff，你想想如果你有1万个提交以后就会变成什么？你这个commit message变成slope了？如果一万个家都是I some cool stu对，那他如果是这样，commit message这个特性就不应该存在。对你不要叫commit message，就从这个V一到V2就行了。然后你会在各种时候用不同的commit message。比如说你这个版本要都收尾了，你可能会有一个版本一个commmanager会讲我要finalize这个版本。甚至Git也允许你在不改任何代码的时候，就这个we和we are是完全相同的时候再做一个提交。

仙1-107老师



给导师原班你了 bilibili

Thinking...

这个 commit 不太符合 best practice. 主要问题在提交信息 (其次代码本身也有问题)。

问题分析

方面	现状	Best practice
提交信息可读性	I did some cool stuffs — 完全看不出做了什么	说明「做了什么 + 为什么」。让未来的你和协作者能看懂
格式规范	无统一前缀	建议遵循 Conventional Commits: feat: / fix: / refactor: 等
信息粒度	只是一个新文件, 粒度过小	一个 commit 应是一个逻辑完整的变更, 但新增文件本身意义不明
代码内容	int main() {} 没有 include, 没有 return	与 commit 规范无关, 但属于质量问题的信号

举例, 更好的写法:

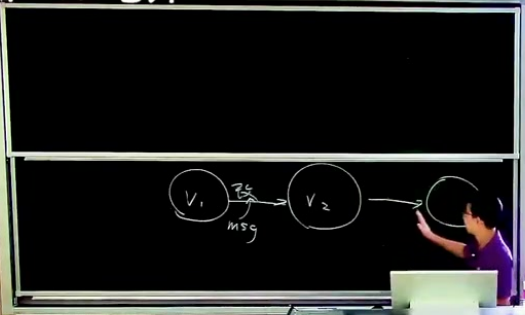
feat: add hello program skeleton

画面 00:41:00

[00:41:15] 这件事情可以吗？对吧？你看他拒绝提交，你的这个空的就没有修改的，但是agent可以。我让他做一个提交，然后。要求他在不做这个的情况下提交，你看他知道他有知识，所以其实你你们你们需要惊人的注意力。

[00:41:57] 然后尝试问从理论上说应该可以，对吧？比如说我就要有一个，比如说我就说我要发布一个版本了，这是一个你就想但这个不是best practice，所以尽量不要做，你想要就要故意做一个这样的空的提交。然后确实看到了一个empty me确实是空的。你看在什么时候有用，可以在无改动时产生一次叫触发CI或者扭打，对吧？

仙1-107老师



给导师原班你了 bilibili

On branch master
nothing to commit, working tree clean
bccfe44 I did some cool stuffs
Took 0.0s

Thinking...

\$ cd /tmp/hello && git commit --allow-empty -m "Empty commit" && git log --oneline -3

[master a227ea4] Empty commit
a227ea4 Empty commit
bccfe44 I did some cool stuffs
Took 0.0s

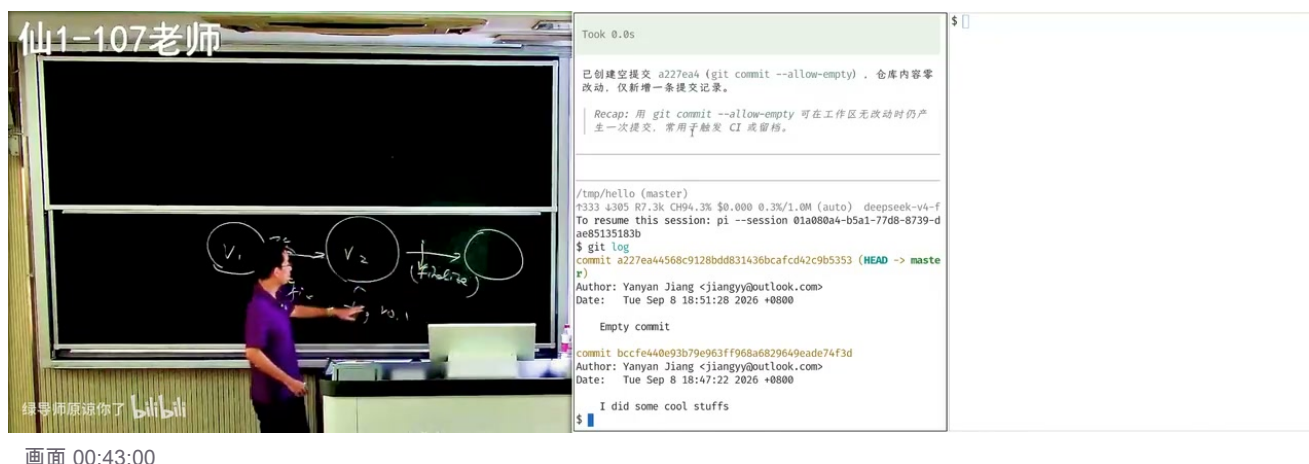
已创建空提交 a227ea4 (git commit --allow-empty) . 仓库内容零改动, 仅新增一条提交记录。

Recap: 用 git commit --allow-empty 可在工作区无改动时仍产生一次提交, 常用于触发 CI 或留档。

/tmp/hello (master)
#333 4305 R7.3k CH94.3% \$0.000 0.3%/1.0M (auto) deepseek-v4-f

画面 00:42:00

[00:42:30] 当我要有一个release，当其release可以把tag打在这儿。但比如说这是一个bug fix，在上一次的时候我我修完了最后一个bug，然后我就得到了一个我可以release的版本。当然Git是可以在这放一个tag的对吧？比如说V0.1放一个tag说这是V0.1，但是我也可以强行做一个提交，说我要翻个来对吧，然这是没有改动的。Final as就是说V0.1 free所有的东西，那这就是一个没有改动的那我就把这种bug fix的提交和finalized的提交给分开了。



画面 00:43:00

[00:43:08] 那这件事情在偶尔的情况下还是可以允许的，但你不能显然不能创建大量的这样的空的提交，对吧？所以你会看到在g里面，虽然它模型ok你学了git push git pull，你就可以用Git了，都会用了。那其实真正水面下的冰山是各种各样的best practice，而这每一个best practice其实都是血和泪的教训。

[00:43:41] 你反过来想，为什么message要这样写？你转手问AI你可以跟更有智觉的模型讨论一下，这样做有什么好处？我觉得很多时候模型它的表现比我好，就可能这个知识我是知道的，内化在我的脑子里面。然后你要给我比如说给我2个小时的时间去想，我可能也能给你写一个非常好的为什么要做这件事。但是我的智力是有限的，我的精力是有限的，我的注意力也是有限的。比如说我现在在上课的时候，我为了专注的和大家上课，我大脑里面做思考的那部分，其实我就有点不够了，我容量都有点不够了。所以我很多时候我觉得我上课讲授一个概念的时候，我讲的是三宝的，我讲的不是最好的。

[00:44:30] 如果你回头把这些问题，比如说为什么commit message要这样写，你去跟比如说chat g机聊，就像我刚开始那个说对吧？我求助一个我无法掌控的东西，但他知道这个世界上所有的对吧？萨姆号知道世界上所有的东西，他能很好的把这些概念给你给你讲讲清楚。然后这些约定俗成的best practice，比如说我来做一点不是best practice的事情，比如说这是你这你这是你们非常常见的事情，这个是可以编译的。

08 暂存区、提交粒度与 .gitignore

[00:45:12] Hello, 这时候你有一个untracked file，然后你们在tutorial里面看到有一个命令很好叫git add .。这样你如果有好多好多的文件，比如改了三五个文件，你不需要git add a.c b.c c.c了，对吧？你是git add .好，然后好你就把它提交上去了。Git是不阻止你做这件事情的对吧？这是完全合法的。我再问问。

[00:45:49] 那我就问问git add有什么不符合best practice的吗？他他会他会看一看，按严重程度排序，那还是很后面的提交信息毫无信息量。对，他已经对在开卖了主观费废话，这就比较好，他还会告诉你应该怎么做，对吧？用析使句现在时描述变更动机，例如fix off my one in H lunch，你的心里面有这种不好的practice，我现在fix了，然后常用的约定comm能commits以前缀也值得考虑，但这个就是你可以更多的项目里面看到这样的commit方式。

仙1-107老师

绿导师原班你了 bilibili

画面 00:46:00

---reflog/tags---
Took 0.0s

有几点不符合 best practice, 按严重程度排序:

1. 提交信息毫无信息量
- slop - 没说明改了什么
- Empty commit - 提交信息本身在说“这是个空提交”, 但 commit message 应当描述内容/原因, 而不是操作本身
- I did some cool stuffs - 主观废话

Best practice: 用祈使句/现在时描述变更动机。例如 Fix off-by-one in strlen check. 常用约定 (conventional commits) 如 feat, fix: 前缀也值得考虑。

2. 提交粒度 / 内容混杂

```

...
b0cfe44 hello.c 2 行改动
04f4f85 hello 15KB 二进制(编译产物)
...

```

- hello 是编译产物, 根本不该进版本库 → 应加 .gitignore, 用构建系统产出
- hello.c 只改了 2 行 (大概率是加换行/尾随空白), 单独一个 commit 意义不大: 把“修复空提交”这类事情和真正改动合并更干净
- 把二进制和源代码分开两个 commit 是对的, 但二进制本身就不该提交

off-by-one in strlen check. 常用约定 (conventional commits) 如 feat, fix: 前缀也值得考虑。

2. 提交粒度 / 内容混杂

```

...
b0cfe44 hello.c 2 行改动
04f4f85 hello 15KB 二进制(编译产物)
...

```

- hello 是编译产物, 根本不该进版本库 → 应加 .gitignore, 用构建系统产出
- hello.c 只改了 2 行 (大概率是加换行/尾随空白), 单独一个 commit 意义不大: 把“修复空提交”这类事情和真正改动合并更干净
- 把二进制和源代码分开两个 commit 是对的, 但二进制本身就不该提交

3. “Empty commit” 这个提交本身就是 anti-pattern—通常意味着你在修复 --amend 弄错的状态或补救历史, 一般应该 rebase/amend 而非另起一个空提交。
4. 没有关联则上下文: 无 issue 引用、无说明文字。若仓库有 issue 跟踪, 建议引用 (Closes #12)。
5. 小点: 分支还叫 master (现代项目多改 main), commit 未签名: 不过单机演示这些都无所谓。

建议的 rewrite (历史还没推送, 可以放心改):

画面 00:46:00

[00:46:35] 好, 第二个提交力度和内容混杂。他开始提醒我了, 这个还是你看这是首要的对吧? 提交信息毫无形象, 这最重要的问题。然后提交力度hello是编译产物, 根本不应该进版本库, 应该加进.gitignore用构建系统产出。所以Git的使用方式和你们的这种作业V1、作业V2还是不一样的。因为你的作业V一里面那个ES一定在里面, 作为V2你的E也在里面, 但是Git它为了防止这个版本库的膨胀。

[00:47:17] 尤其是如果你有你这个项目里面有1万个文件, 有100个合作者的时候, 每一个合作者的环境都是不一样的, 对吧? 你想每个环境都是不一样, 他们build出来的东西也是不一样的。你们可能不知道你们的及你们的编译, 每一次那个文件都可能是, 比如说带一个日期的, 它甚至带hash的。但你本机的指纹的, 它每一次这个二进制文件可能都是不一样的。然后如果你把这个东西放进去了, 下一次就会产生冲突, 对吧? 你在因为你你这里有一个hello, 对方有个hello, 哪怕是一样的源代码编译出来, 它都可能不一样。更何况如果是不一样的源代码编译出来的hello对吧? 然后呢会产生一个二进制的冲突, 而二进制的冲突是很难合并的。

仙1-107老师

绿导师原班你了 bilibili

画面 00:48:00

off-by-one in strlen check. 常用约定 (conventional commits) 如 feat, fix: 前缀也值得考虑。

2. 提交粒度 / 内容混杂

```

...
b0cfe44 hello.c 2 行改动
04f4f85 hello 15KB 二进制(编译产物)
...

```

- hello 是编译产物, 根本不该进版本库 → 应加 .gitignore, 用构建系统产出
- hello.c 只改了 2 行 (大概率是加换行/尾随空白), 单独一个 commit 意义不大: 把“修复空提交”这类事情和真正改动合并更干净
- 把二进制和源代码分开两个 commit 是对的, 但二进制本身就不该提交

3. “Empty commit” 这个提交本身就是 anti-pattern—通常意味着你在修复 --amend 弄错的状态或补救历史, 一般应该 rebase/amend 而非另起一个空提交。
4. 没有关联则上下文: 无 issue 引用、无说明文字。若仓库有 issue 跟踪, 建议引用 (Closes #12)。
5. 小点: 分支还叫 master (现代项目多改 main), commit 未签名: 不过单机演示这些都无所谓。

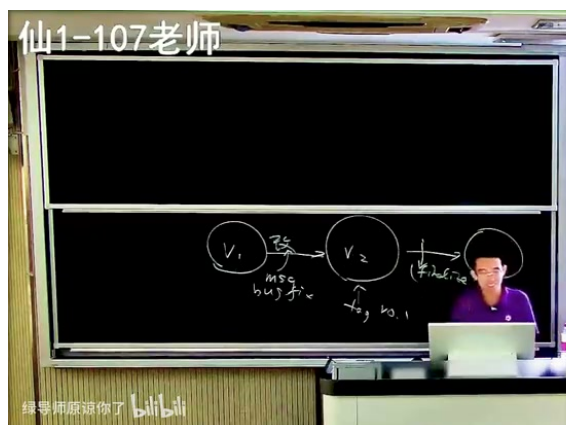
建议的 rewrite (历史还没推送, 可以放心改):

画面 00:48:00

[00:48:05] 你想如果我编译出来的东西不是这种程序编译出来的, 如果我生成的图片? 当然图片是一个比较特殊的这个东西, 对吧? 图片怎么怎么合并呢?

[00:48:18] 合并就是有有重复OK, 所以这边有很多很多的best practice。然后我看到他对他可能会出乎意料, 我不知道他他会会发生什么。支付网断了, 没有, 还花了一些比较长的时间来思考网没有断开, 他说丢弃空提交和编译产物两段历史, 新建main分支重写为两个有意义单一值责的提交。好, 他还贴心的发现历史没有rete, 也就是说我的仓库没有和另外一边同步, 不会有别人。因为如果别人也持有的话, 那就是一个比较糟糕的事情了。没有别人, 所以他把整个这个项目就给改了。

[00:49:25] 你看如果你的注意力注意到，你发现它换了一套这个commit的方式，然后它会有一个这样的headline对吧？Headline的message，然后在headline的message的后面还有一些，然后这里面还有好多好多好玩的。比如说如果你在这里有一个什么写一个什么fix的景3，比如说你在这个message写一个fix的并3，然后GitHub会自动把你的这个issue给close。这里面有很多很多好玩的，你只有反复的使用，你才能获得一个初学者和有一定经验的人的区别。对，当然现在是agent时代，所以你们都是agent native的人，你们肯定也不会用古法。我觉得是这种什么Git status这种都已经成为了古代手艺了。你们现在一定都是用agent对吧？



画面 00:50:00

获取下一级知识

学会追问，学习才进入正轨

绝大部分人都倒在了“获取下一级知识”的路上。

- ◆ “能运行”以后：为什么别人还要写 .gitignore？
- ◆ “文件都在”以后：为什么换一台电脑就编译不了？

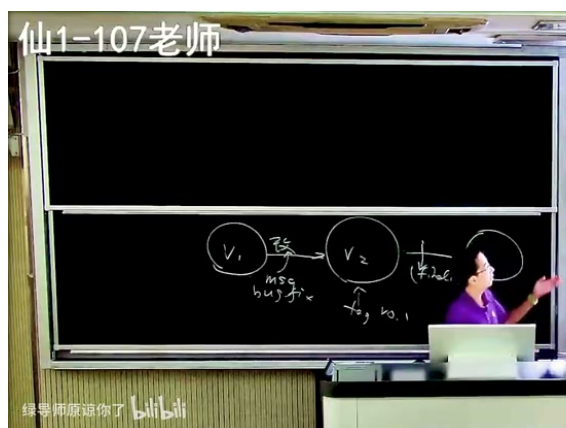
Best Practice 从哪里来？

软件和系统都有**约定俗成**的 best practice，而每一个都是血的教训，或者对旧设计忍无可忍的产物。

- ◆ 把源码和编译产物一起提交：源码改了，产物忘了更新——同一件事有了两份互相矛盾的状态。
- ◆ 把自己的绝对路径写进配置：在我电脑上能跑，到了同学那里就失效。
- ◆ “目录里有什么”和“项目需要共同保存什么”，是两个问题。

2/8

[00:50:30] 说你不需要知道我具体的命令是什么，但是你只要有概念说我的Git能干什么，你就可以让他用符合best practice的时候干。当然这带来一个问题是你们其实根本就分不清你是真的就你自己能干，还是你做就干不了。但也许我觉得这也不重要，所以为了加增加大家的学习的效率，其实可以这样？



画面 00:51:00

做任何 git 操作之前，都这个文件检查变化，并为我（新手）解释是否符合 best practice，为什么，以及相关的概念。

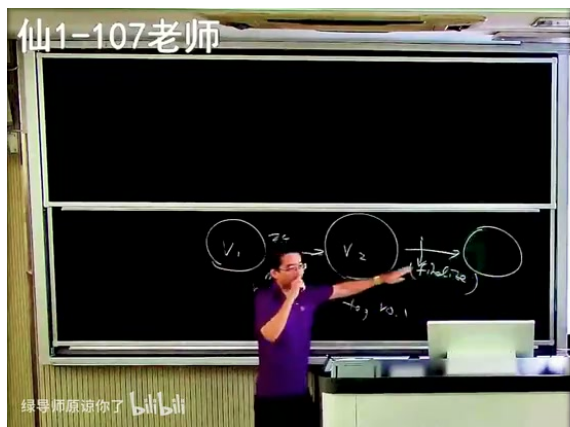
AGENTS.md [New] 1L, 153B written

[00:51:00] 在做任何k操作之前都要逐个检查文件的变化。比如说我来随便试一个是一个不能编译的代码，对吧？这个会编译不能编译好，我让他做一个提交，我也不知道模型会给我什么。你看他很聪明，他比你聪明，你在改作业的时候，我觉得我们都会有这种倾向。就改一个字不要紧的，就改了就提交，对吧？你一定会这样想的。但是模型有工程，就是他见过很多好的工程项目，也读过很多软件工程的书。所以他知道，当前不建议直接提交，然后他还给你修好了，还他给你解释，对吧？

[00:52:00] 可能有几个prompt，比如说我有一个我的有一个system prompt，大概可能比如说要说中文或者我在上课。所以你可能要讲的更详细一点，这样的prompt然后以及他给了很多很好的解释，对吧？为什么它不符合Git？提交应当是一个完整字治的快照提交编译不过的代码，相当于让仓库处于坏状态，而且你看它还有更多的。

[00:52:27] 这是我都没有想到，这就是我我知道，但是我在上课的时候，即兴的时候，我是想不到他的，对吧？如果真的要人去去这个git bisect，还有Git它还有一个很有意思的功能叫git blame，我很喜欢这个功能，比如说hello点C对，然后他会告诉你每一行代码都是在哪一次提交，哪一个地方改的。也就是说当我有一个哪怕是一个线性的这样的历史的时候，我都可以干很多有趣的事情，对吧？你看一下子AI agent的一句话就把Git给打开了。

09 定位问题：blame、bisect 与自动检查



仙1-107老师

绿导师带你学 bilibili

画面 00:53:00

```

error: 'a' undeclared → a 没被声明
error: expected ';' → 语句缺分号

```

为什么不符合 best practice?

- Git 提交应当是一个“完整、自治”的快照。提交编译不过的代码，会让仓库处于“坏状态”。
- 如果有人之后用 git bisect (二分定位 bug) 或 git checkout 回溯到这次提交，拿到的是一个编译失败的版本，排查问题会很痛苦。
- 一条广泛接受的原则：**每次提交的代码应当能通过构建/测试**（至少主分支如此）。CI（持续集成）工具也会基于这条假设工作。

相关概念：a 这里的报错本质是—C 语言里每一条语句必须以 ; 结尾，而且 a 本身既未声明为变量，也不是表达式，属于非法语句。改动看起来是“打字到一半”，可能是误操作。

文件 2: AGENTS.md (未跟踪) — 可以提交，但建议先想清楚

内容是一段对协作方（比如 AI 助手）的行为约定：“做 git 操作前这个检查变化并解释”。

为什么符合 best practice?

- 把项目约定作为文档放进仓库，新加入协作的人都能看到，避免约定只存在于某个人脑子里——这本身就是好习惯。
- 很多仓库会把 AI 协作规范（如 .cursorrules, AGENTS.md）版本化，方便团队共享。

需要你决定的两点：

```

default: off)
-w          ignore whitespace differences
--[no]-ignore-rev <rev>
              ignore <rev> when blaming
--[no]-ignore-revs-file <file>
              ignore revisions from <file>
us line differently
--[no]-color-lines
              color redundant metadata from previo
--[no]-color-by-age
              color lines by age
--[no]-minimal
              spend extra cycles to find better ma
tch
-S <file>    use revisions from <file> instead of
calling git-rev-list
--[no]-contents <file>
              use <file>'s contents as the final i
mage
-C[<score>]  find line copies within and across f
iles
-M[<score>]  find line movements within and across
files
-L <range>   process only line range <start>,<end>
> or function :<funcname>
--[no]-abbrev[<n>]
              use <n> digits to display object nam
es

$ git blame hello.c
^9ce44b0 (Yanyan Jiang      2026-09-08 18:58:30 +0800 1) int m
ain() {
00000000 (Not Committed Yet 2026-09-08 19:12:16 +0800 2)  a
^9ce44b0 (Yanyan Jiang      2026-09-08 18:58:30 +0800 3) }

```

[00:53:08] 我们可以做二分查找。大家都知道二分查找吗？你们可能会在保研的时候或者工作面试的时候，去要去手写一个二分查找。老师说如果你有一个很长的数组，对吧？你也可以把这个Git提交的历史想象成是一个很长的数组。然后你现在在当前的版本上发现一个bug，但这个bug可能不是由我的代码引起的，可能是过去的某一个地方引起的。你这时候有一个测试用例，或者有一个任何一个能够检查出这个bug的东西，你就可以对整个的Git的历史记录做二分查找，对吧？

[00:53:45] 因为本质上这个Git的历史是有一些提交是ok的，有些提交是不ok的。然后应该是在某一刻引入了一个bug以后，从某一个时刻开始直些提交就都不能通过了。但是在这个之前的每一个提交应该都是ok的。所以这不就是一个在所有的后缀都是零，后缀都是一的数组里面找第一个一，对吧？至于二分查找能找的事情，你可以做做做二分，然后你还可以git blame，你可以知道是谁在你知道错了以后？知道错了以后你可以知道是谁改了这一行代码，然后发生了什么。

[00:54:31] 对然后对他有他有很多软件工程的实践，虽以软件工程，我一直觉得可能不需要在课上讲那么多的东西，你要把时间留给你们，你们自己在实践当中由agent来告诉你，每次提交的代码应该能够通过构建和测试，这是这其实是一个非常基础的要求。大家也会觉得ok k应该是这样，但是在你反复的强化潜移默化以后就ok了。对，然后他就说a ent的MD是可以可以提交，这是符合这个best practice，我就不再不再做了。所以好消息是我以前在M之前，一般来说有新同学进组都会有一些高血压时刻。就是使用Git的best practice。因为比如说我们要做一个研究项目，你欢迎你加入这个研究项目。你去开一个Git Rio，然后你大概你第一周可能会有一个非常简单的任务，对吧？

仙1-107老师

error: 'a' undeclared -> a 没被声明
error: expected '}' -> 语句缺分号

为什么不符合 best practice?

- Git 提交应当是一个“完整、自治”的快照。提交编译不过的代码，会让仓库处于“坏状态”。
- 如果有人之后用 git bisect (二分定位 bug) 或 git checkout 回退到这次提交，拿到的是一个编译失败的版本，排查问题会很痛苦。
- 一条广泛接受的原则：**每次提交的代码应当能通过构建/测试**（至少分支如此）。CI（持续集成）工具也会基于这条假设工作。

相关概念：a 这里的报错本质是—C 语言里每一条语句必须以 ; 结尾，而且 a 本身既未声明为变量，也不是表达式，属于非法语句。改动看起来是“打字到一半”，可能是误操作。

文件 2: AGENTS.md (未跟踪) — 可以提交，但建议先想清楚

内容是一段对协作方（比如 AI 助手）的行为约定：“做 git 操作前这个检查变化并解释”。

为什么符合 best practice?

- 把项目约定作为文档放进仓库。新加入协作的人都能看到，避免约定只存在于某个人脑子里——这本身就是好习惯。
- 很多仓库会把 AI 协作规范（如 .cursorrules, AGENTS.md）版本化，方便团队共享。

需要你决定的两点：

画面 00:55:00

[00:55:37] 搭一个架子，然后把一两个东西跑起来。好，然后一般来说我都会在打开那个Git项目以后，进入一个高血压环节。比如说我一看到比如说disk或者log会在这个会被交上来，对吧？Lolog也是生成的文件，所以它不应该叫个人的配置。比如说v你个人的设，甚至这个vs code里面会有你的什么C盘下的什么东西的那样的一个硬编码的路径，显然在任何一个其他地方都是没有办法办法复现的。

[00:56:11] 后但是更高起量的是我可以在审稿的时候看到一些可以处于active状态的API。我当我发现这个api是deep k的时候，我就知道这应该是一个中国兄弟。以及你没有ma克的同学不经意之间会提交上来这个.DS_Store这这是这是Git的一个特性对吧？如果这是没有一个no对，没有一个，然后我可以在苹果系统上，你会就是如果你用finder去打开一个目录，它会自动留下一些索引，大概是你把哪些子目录展开，这些放在一个叫.DS_Store的文件里。但这个文件因为是点开头的，所以它是。默认是隐藏的。你看我的I还还很贴心的给他一个苹果的图标，苹果图标，所以你是看不到的。然后你看不到的时候你会git add .，然后git add .就糟糕了。

仙1-107老师

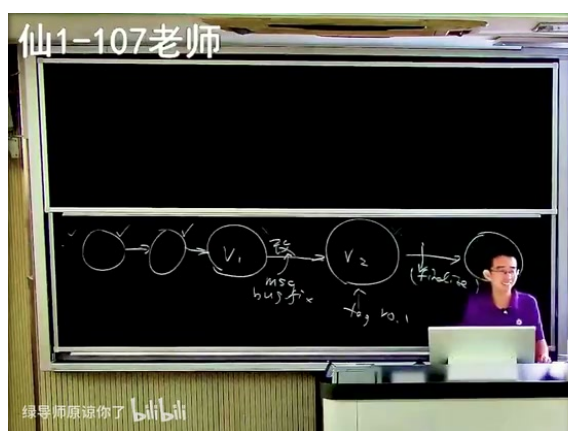
```
$ eza --git --group-directories-first --icons=auto
AGENTS.md  hello  .DS_Store
$ eza --git --group-directories-first --icons=auto -l
.rw-rw-r-- 153 jyy 8 Sep 19:10 -M AGENTS.md
.rwxrwxr-x 16k jyy 8 Sep 18:54 -I  hello
.rw-rw-r-- 19 jyy 8 Sep 19:10 -M  hello.c
$ eza --git --group-directories-first --icons=auto
AGENTS.md  hello  .DS_Store
$ eza --git --group-directories-first --icons=auto -al
.rwxrwxr-x 0 jyy 8 Sep 19:16 -I  .git
.rw-rw-r-- 7 jyy 8 Sep 18:58 --  .gitignore
.rw-rw-r-- 153 jyy 8 Sep 19:10 -M AGENTS.md
.rwxrwxr-x 16k jyy 8 Sep 18:54 -I  hello
.rw-rw-r-- 19 jyy 8 Sep 19:10 -M  hello.c
```

画面 00:57:00

[00:57:14] 你看这个点点store会被Git追踪，当然这也是合理的。因为点有可能是一些比如说点配置文件，点GitHub，workflow这些约定俗成的东西，所以它默认带点的都会放进来。但这个临时文件就有一点糟糕，因为你你你可能会经常的变，然后它就不应该被提交进来。但又有一些这个规则又比较复杂，比如说不是所有的generated fire都不应该提交，比如说你可能有个package lock点这这对吧？然后你可能会generate一个header，然后这header确实是可以generate，但是你把它放进来就可以方便编译。别人可能在不需要下载很多依赖的情况下，依然可以编译这个项目，对吧？你可能用另外一套工链去生成的这个header，然后这个header但是你可以在一个没有这个工具链的另外一个地方去去编译它。所以也没有绝对的规则。

[00:58:10] 我们看过很多软件，因为我自己是做软件工程研究的一个非常标准的软件工程研究，就是比如说去研究你的Git的这个项目满不满足这些best practice。然后在不满足的时候给你一个warning。比如说像d store这种东西，它就需要一个规则对吧？后然后好多好多规则，后最后做一个小工具，在Git提交之前告诉你ok不ok但所有的这些东西都被大模型杀死了，因为大模型可以如此耐心的给你检查，还给你一个emo让，你开心一下？缓和你的情绪。

[00:58:47] 有的时候就比如说我作为导师，我今天把学生叫到办公室，然后说，我肯定第一个我肯定高血压了，第二个我还得装作nice一点，对吧？我会说好，你提交的这个项目不太符合软件工程的实践，然后这样然后同学生在在听的时候，他也觉得我不满意，对吧？就不管怎么样，我在纠正他，无论我的情绪多么正常，我只是告诉他一个，我教授他一个知识，这个时候学生总是有压力的。对，毕竟是老板批评我了，我做了一件我不该做的事，然后这个suppose可能是应该知道的，所以AI你看真的解放了这个同学们，那个你无法控制的力量。Ok而且真的我觉得AI很强。我最近在一个更严肃的就一个更严肃的项目，我希望把每一个细节都做好的时候，我面临一个选择。



画面 00:59:00

进组以后的高血压时刻

我和许多研究生新生都说过

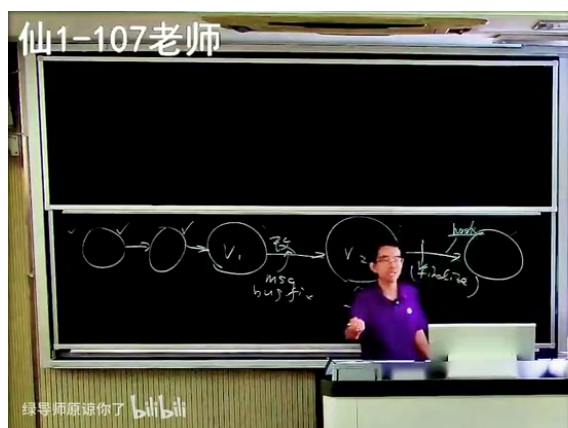
- 生成的文件不提交 (dist/, log,
- 个人的配置不提交 (.vscode 中的个人设置等)
- ♦ 判断的是用途：团队共享、可移植的配置可以提交，不能只看目录名。
- 密钥不能提交 (审稿的时候见过好多)
- 临时文件不提交 (.DS_Store,
- 规则是活的：package lock, generated header, ...

忽略不等于遗忘

- ♦ `.gitignore` 不会让已经被跟踪的文件自动退出版本管理。
- ♦ 密钥一旦泄露，删掉当前文件也不能收回别人拿到的副本：应先撤销或轮换凭据，再处理历史。[GitHub 指](#)
- 查

4/10

[00:59:47] 对，大概是我们在Git提交之前，我们有一个记者叫大家用过吗？各种各样的hook 钩子，你可以ho程序的一个函数调，然后在这个函数调用结束的时候干一件事。你也可以在你的agent里面加入一个ho在，某件事发生的时候干一件事。比如说我有一个ho会，在agent完成需要我输入的时候，它会播放一个声音叮一声。然后你现在也可以直接就是你你你可以直接让他说什么，帮我配置一个全局的ho在agent需要输入的时候叮一声，提醒完全没问题。你在你的任何一个Claude Code，你的codex，你的派，它都支持这样的钩子。



画面 01:00:00

大写还是小写？

真实项目中的一个选择

```
[TYPO-CHECK] Docs should be typo-free
v.s.
[typo-check] Docs should be typo-free
```

Check-rule 的名字应该是大写还是小写？

直觉也值得追问

我的直觉是“大写似乎更好，但不是极强”，但不太说得原因。

- ♦ 先想一想：方括号里的字符，是一句话的一部分，还是一个可以被引用的规则名字？

5/10

[01:00:42] 然后我可以在Git的pre-commit hook上面做一些检查。因为可能这个项目的工程标准会更高一些。那么呢我就面临一个我应该选上面还是应该选下面。比如说我要给我的各种规则，可能有我有几十个这样的checking rules，然后我要给这个规则命名，那我应该选这个大写还是小写呢？

[01:01:08] 然后从首先从使用的角度来看都没有问题，就是两个都是好的。只要你只要我这里没有typo对吧？我这句话里没有type就是ok的。但到底是大小写还是小写，两难的问题。然后你在在学习和生活的过程中，你又面临无穷多这样两难的问题。如果你的每一个问题都稍微问一句，你就会有很多很多的长进和知识。我看一下。诶他在他在处理呢。

[01:01:58] 他说好啦。有声音吗？没有，我好像听到声音了，对吧？很轻的一声，再来一次。

仙1-107老师

绿导师原班你了 bilibili

```

pw-play (PipeWire, 已检测可用) 播放: 找不到播放器时返回终端 BEL \x07.
- 不打扰: spawn 后 unref(), fire-and-forget. 绝不阻塞或拖住 pi 进程: 检测, 合成都只做一次并缓存。
- 只响应真实需要输入的时刻: 每次你发一条消息, 它跑完一轮就响一声; 启动时, agent 仍在运行时都不会响。

你需要做的

在 pi 里执行一次 /reload (~/.pi/agent/extensions/ 会被自动发现并热重载), 之后每轮 agent 干完活就会听到“叮”。

想调整音高/时长/音量, 改文件里的 1046.5 (Hz)、dur、0.3 (音量) 后再次 /reload; 想停用直接删掉该文件即可。

Recap: pi 的扩展机制支持 pi.on("agent_settled", ...) 事件钩子 (配合 ctx.isIdle() 精确判定"agent 已停, 等待输入")。扩展放在 ~/.pi/agent/extensions/*.ts 为全局生效, 可 /reload 热加载。声音用 Node 现场合成 WAV (正弦+指数衰减+铃声) 再走 pw-play 播放。无系统音频依赖: 测试时先验证了 PipeWire 通路, 再用 fake pi API 加载扩展验证了注册与触发逻辑。

Reloaded keybindings, extensions, skills, prompts, themes, and context files

/tmp/hello (main)
#12K #9.9K R203k CH99.9% $0.005 2.4%/1.0M (auto) deepseek-v4-

```

画面 01:02:00

[01:02:20] 没有，anyway，但他你看他意识到这个，意识到我的意图了，然后你让他修就行了，我就不让他修了。在这里面AI给了，我让GPT解释一下，然后你看他这个typo check，他说它是一个固定规则的标识，然后他还从这个C里面给了个一些引用，这个就是他分析我的心理，为什么我会觉得这个全大写会更强一点？然后如果你们能在学习的过程中，比如说在学习Git过程中多问几句，把每一个细节都做好，那你们一定会成为a lot better person这样。

10 Git 的底层数据结构

仙1-107老师

绿导师原班你了 bilibili

这里，我会选 **TYPO-CHECK**

用字形提示不同的角色

- ◆ [TYPO-CHECK] 是固定的规则标识: Docs should be typo-free 是解释规则的自然语言。大写让两者在视觉上更容易分开。
- ◆ 讨论“违反了 TYPO-CHECK”时，这个短名字也更像一个明确的引用。这是此处的设计判断，并非“大写总是更可读”。

约定赋予字形含义

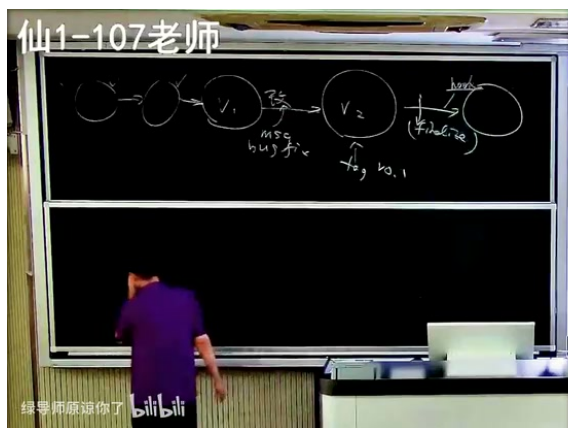
- ◆ 一个类比: RFC 8174 明确, MUST 等关键词仅在大写时承载约定的特殊含义, 避免与普通行文混淆。
- ◆ 它没有规定 check-rule 必须大写。项目若已有小写规则体系, 或工具要求小写, 应优先保持一致。
- ◆ 所以“大写似乎更好, 但不是极强”: 这里有让规则标识更醒目的理由, 也有比这点偏好更重要的上下文。

画面 01:03:00

[01:03:07] 对，好，行，那么接下来你要开始学习Git了，吧？首先你ag点MD也有了吧？Agm d也有了，那么接下来就是要想要学一学Git到底是怎么工作的。Git的底层原理，当然我我可能会推荐大家去看，

我在操作系统课上面讲Git的那那一段，我就没有必要再讲一遍了，我就讲一个头，讲一头是什么呢？是Git是一个持久化的数据结构，所以它就是一个数据结构。

[01:03:49] 那Git持久化的是什么呢？目录数对吧？因为我们说就是快找不停的版本，版本一版本2版本三、版本四，那你就从这个最基础的想法出发，你就知道如果我要存一个目录数，比如说杠对吧？比如说hi比如说A点TXT那么B点TXT对吧？还有一个B点TXT好，那有可能是在这里面还有一个，比如说还有一个目录叫D然后D里面有无穷多个文件。这个太多了，这个doesn't matter。



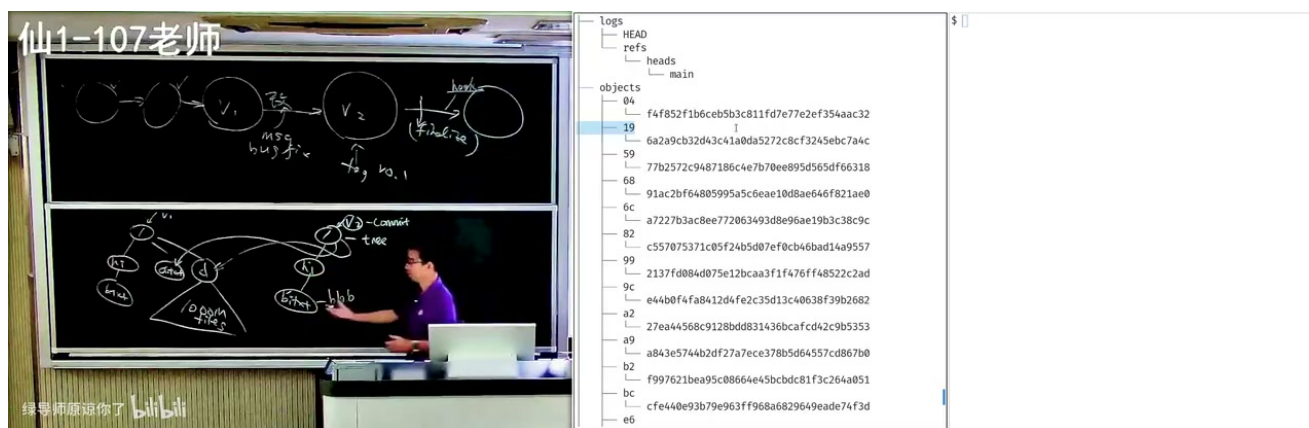
画面 01:04:00

[01:04:26] 好，我现在有一个很大，相当于我有一个非常大的项目数，我今天这个已经快到了。比如说这就是一个V1，我现在要创建V2，我要创建V2，我需要把所有的文件都复制一份吗？我不需要。比如说我现在改的是B点TXT好，我现在这个B点TT改了，那我肯定是需要再创建一个B点TX7的新版本，对吧？

[01:05:00] 新版本然后如果我想要以最小的代价记下一整棵树，我应该怎么办呢？我应该把它到根的路径路径，反正我要我要再复制一份，但是根里面的这个A点TC我就可以用一个指针指过来，对吧？A点这个根下的害我不能指向这儿，因为指向这儿就意味着我要用旧的B点填C所以它的根的路径都要复制一份。那我有一个指针指到这里，然后以及我的这个D我也可以有一个指针指向这里，虽然不太好看，但这就不一样。

[01:05:42] 所以即便你的项目很大，里面有100万个文件，你只要首先目录本身就不可能很深，没有人会去想建一个一千层深的目录，对吧？你也没有必要从这个鉴定式披露的角度，你没有必要去建一个巨深无比的它。只要你每一级索引都是指数级的增加，对吧？所以你很快你就能把所有的类给分完。你只需要创建一个非常小的增量，你就得到了一个版本的快照。所以在Git it里面，在Git里面，它底层大家看到点Git。

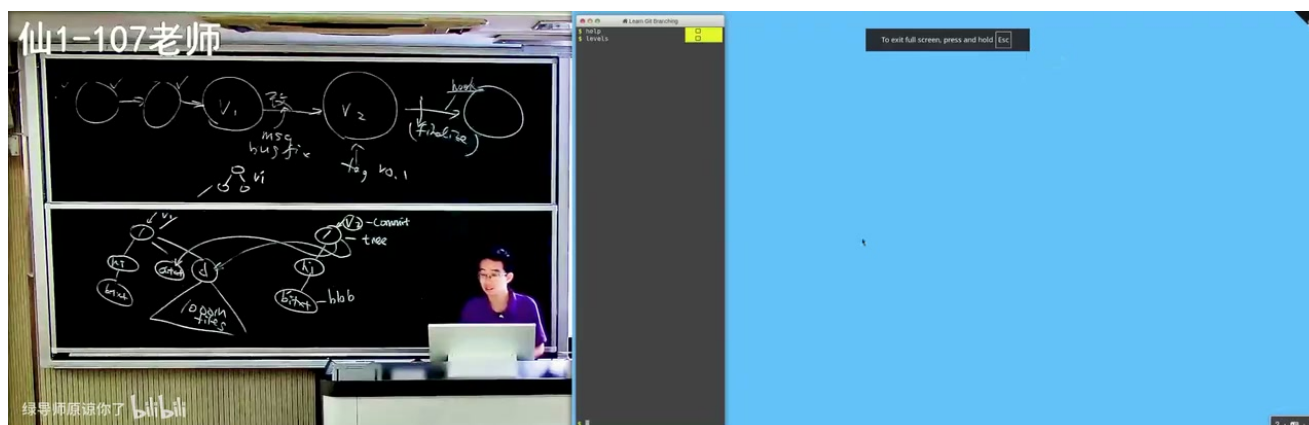
[01:06:24] 开始在点git里面有各种各样的objects。然后这个objects它就是一个普通的文件，你看是带hash索引的。然后就是这种节点，各种节点目录的节点，文件的节点，然后甚至有commit的节点。Commit object它有commit object，然后它有tree object，然后它还有blobject三大类型的对象。Blo就是就是binary，就是字节字节的字节的序列，然后它就是一个数据结构，然后这个数据结构可以不断的向后生长，向后增长，然后这就解决问题了，其实就解决问题了。所以你们如果从1st principle，就一个大学生想要设计出一个这样的东西，是完全有可能的。对，完全有可能的。好，下一个，你能不能就这这是这是纸面上的讲解对吧？



画面 01:07:00

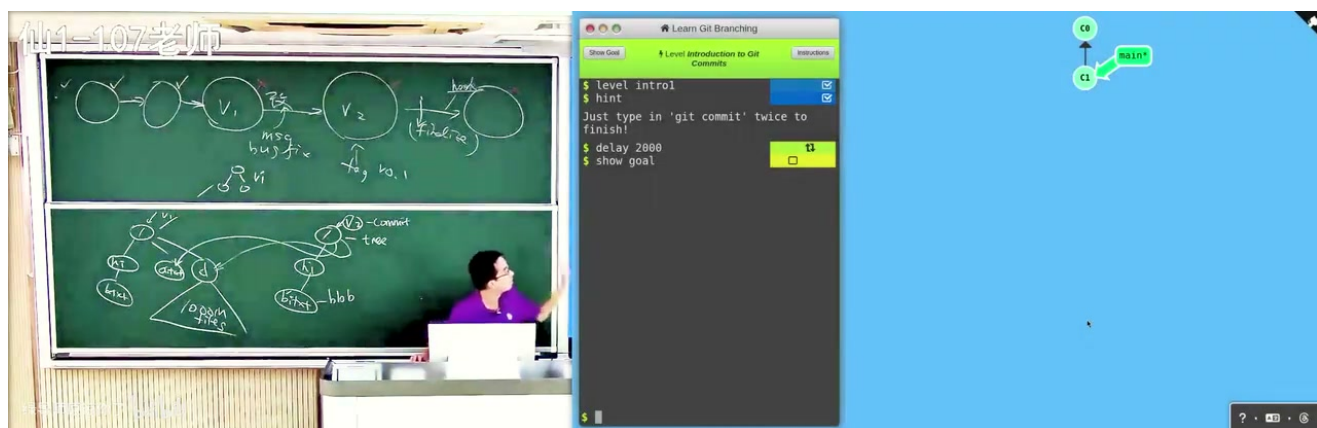
[01:07:27] 我讲完以后你有没有觉得有点不踏实，我们的Git真的是这样实现的吗？老师上课说这个 commit object tree object blob object真的有这样吗？然后如果发生分岔？首先因为我可以这样创建快照，我也可以从这个V-版本创建一个另外的一个tree，比如说到这一个吹V-的一撇，然后我的世界就分岔了。分岔以后合并到底会发生什么，你就自然而然的想想这些问题。根据计算机科学定律，你想要的东西就一定有人做。所以所以所以我开一个浏览器。

11 用可视化理解提交、分支与合并



画面 01:08:27

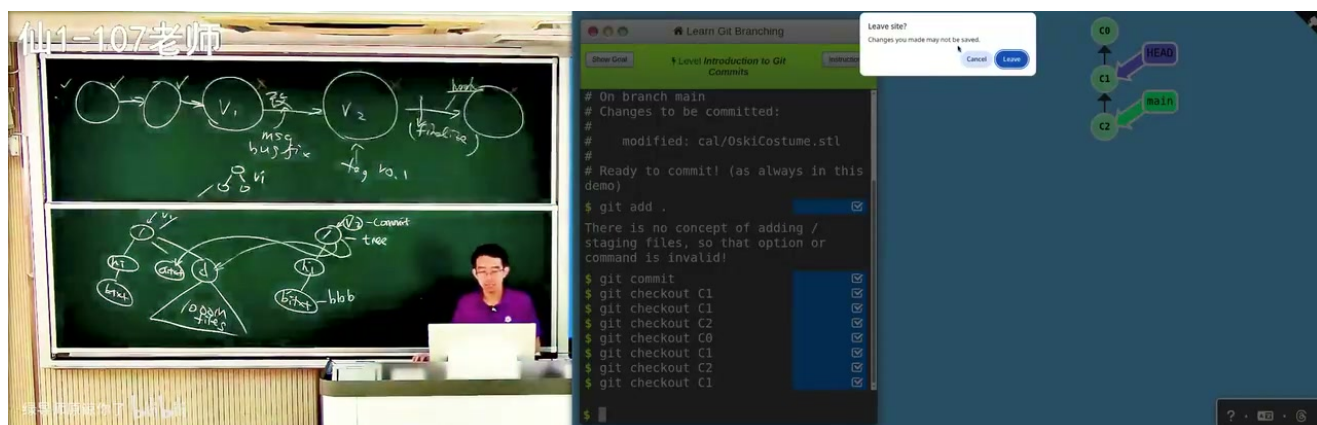
[01:08:30] 我以前上课的时候也会用这个。他是就我相信这是这是这是古法这是古法编程的巅峰之作之一了。它以可视化的方式展示了展示了这个Git是怎么工作的。比如说他甚至模拟了各种各样的命令，你看 git status它都有，现在有一个改了的。



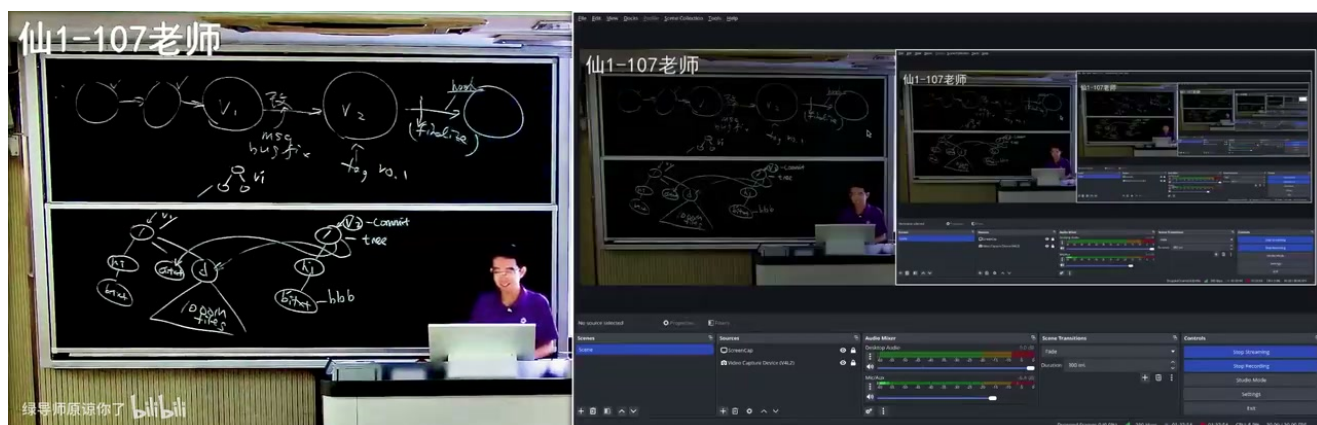
画面 01:08:47

[01:09:07] 好，它不需要add。好git commit。看到右边动画了吗？它就多出了一个版本，然后你看它可以git checkout，我点这个版本的时候，它可以看到版本的变化。它用这个方式来教你这个git是怎么工作的。

[01:09:33] 然后到今天到今天我觉得就可以直接Vibe Coding这些东西。虽然虽然可能Vibe Coding的质量还并没有那么好。我做了一个，我直接Vibe Coding了一个。可视化整个Git repository的工具。好，你看它甚至还有一些未提交的代码。你看GPT-6虽然它有视觉能力，但是一遍直出还是有一些还是产生了一些slop的。我给他这样的一个计时词，实现一个gv命令行工具，把一个真实的Git repository你可以使用Git命令，然后把里面主要概念彻底完整的可视化，展示任何对象之前都从文件系统里的真实文件。



画面 01:09:37



画面 01:10:00

[01:10:43] 注意这是和刚才你看到那个learngitbranching.js.org不一样的地方。它是模拟出来的，它建了一个Git的模型。但是你今天你是有办法在真正的就在live的这个Git repository上面直接看到它发生什么。

[01:11:01] 我从真实的文件原始的bi出发展示一步一步如何解析，解析成了什么。然后我要求它它可视化，并且我让它持续监听目录中Git的变化。然后在发生变化后，用动画效果来展示Rao发生的变化。一会儿。What?

[01:11:30] 我有好，我现来把它删掉。你看我直接删除。你看右边的文件没有了，看到吗？右边的文件没有了。然后我们来看一下这棵树，这是main分支，他就像我画的这个一样。

[01:11:58] 你看最早的一次提交是一个I some cool stu里面有一个空的hello点C空的hello点C然后这个空的hello点C你看到它的这个文件是来自于杠tap下的hello下的一个object，这真的是一个object，然后你能看到它的字节序列，开到这个字节序列以后，它会帮你解压缩。然后你发现对于很小的文件来说，这个压缩反而是浪费，对吧？他31字节解压出来一个23字节的東西，但这不要紧，这是一个bloo你看bloo 15这个文本，这是一个bloobject大小是15。然后in然后all你就可以看到这个。然后除了这样的blog以外，还有这个hello的一个二进制文件。你看的158，这是7F12F1个可执行文件的头，然后你也能够看到它以及它也有tree object。然后tree object同样你也可以看到它的磁盘上面的原始的数据，52字节解压出43字节。然后你看这个tree是这样的，644644代表这个文件访问的权限，然后它有hello点C真的是一个。

仙1-107老师

```
$ eza --git --group-directories-first --icons=auto
= AGENTS.md  D hello  @ hello.c
$ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   .DS_Store
    new file:   AGENTS.md
    modified:   hello.c
$ git reset --hard
HEAD is now at 196a2a9 chore: ignore compiled hello binary
$
```

画面 01:12:00

仙1-107老师

```
$ eza --git --group-directories-first --icons=auto
= AGENTS.md  D hello  @ hello.c
$ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   .DS_Store
    new file:   AGENTS.md
    modified:   hello.c
$ git reset --hard
HEAD is now at 196a2a9 chore: ignore compiled hello binary
$
```

画面 01:13:00

[01:13:19] 你看以文本的形式解压完了以后，就以文本的形式存储到hello点C然后你看它的树的结构，by object，mode 644 name hello点c，tree object，然后这个tree object指向了？看到这个杠在这个版本

里面的这个杠，它就有两个文件，一个755，一个644对吧？951个hello，一个hello，点hello是可以执行的。然后这个指针链接指向了对吧？

[01:13:54] 我可以把它拖拖过来一点，链接指向这里指向它这里，指向它这个视觉效果不完美。是因为我没有调，就是one shot直出没有改，我可能稍微第二个让它去掉了一点东西，就这些所有的问题都没有修。Ok好，然后我们可以在这里各种各样的探索Git的方方面面。比如说我来把它拖过来点。好，你现在可以创建分支，我可以让他创建。

[01:14:49] 对，你可以讲清楚一点，我让他创建两个分支。然后你就直接看到动画效果了，就是左右是实时同步的。没关系，你看这里有一个东西叫a refs has下的A然后你的什么是一个分支？分支就是一个文件，叫ref它就是指针对。所谓的分支就是指向这些东西的指针，然后head是指向指针的指针，它的磁盘上是这些字节，然后这个字节就是一个tree，指向了一个tree object，这个东西就是一个tree object。

仙1-107老师

main 基于同一提交 196a2a9 分出 A、B 两个分支

- 分支 A 把首行改成 int main(void) { 并提交 94d2b56
- 分支 B 把首行改成 int main(int argc, char **argv) { 并提交 27e7fd3
- 两处修改重叠在同一行 → 将来 git merge 或 rebase 时必然产生冲突

... *

* 94d2b56 (A) refactor: main 改用 void 参数

| * 27e7fd3 (B) refactor: main 增加命令

| /

* 196a2a9 (main) chore: ignore compiled hello binary

当前停留在分支 B，hello.c 显示为 B 版本。

Recap: 在同一基线创建 A/B 分支，对 hello.c 同一行做不同修改并各自 commit，制造了行级冲突：历史图中 A、B 分支在 196a2a9 处，未合并（可用 git merge A 演示冲突，此处按约定跳过）。

Working

/tmp/hello (B)

952 41.1k R17k CH96.9% \$0.000 0.4%/1.0M ...

画面 01:15:00

[01:15:38] 好，我们来看A。你看A就是一个就是一棵树，然后树里面有一个杠，然后杠里面包括了新的这个新的hello点C你看它是ink make的。然后同样的BB然后B也是这样的，它指向了这样的一个他改过的带R跟C二工艺的视觉，视觉效果不是很好。但我相信我跟他对话几轮以后，应该就能就能搞定了，就得到一个近乎完美的Git的visualization。

仙1-107老师

main 基于同一提交 196a2a9 分出 A、B 两个分支

- 分支 A 把首行改成 int main(void) { 并提交 94d2b56
- 分支 B 把首行改成 int main(int argc, char **argv) { 并提交 27e7fd3
- 两处修改重叠在同一行 → 将来 git merge 或 rebase 时必然产生冲突

... *

* 94d2b56 (A) refactor: main 改用 void 参数

| * 27e7fd3 (B) refactor: main 增加命令

| /

* 196a2a9 (main) chore: ignore compiled hello binary

当前停留在分支 B，hello.c 显示为 B 版本。

Recap: 在同一基线创建 A/B 分支，对 hello.c 同一行做不同修改并各自 commit，制造了行级冲突：历史图中 A、B 分支在 196a2a9 处，未合并（可用 git merge A 演示冲突，此处按约定跳过）。

Working

/tmp/hello (B)

952 41.1k R17k CH96.9% \$0.000 0.4%/1.0M ...

画面 01:16:00

[01:16:13] 好，我现在你说我说。我可以让你给我展示merge conflict的状态，这时候你看就很正常的很正常。你在两个开发者在开发的时候，对吧？你在假设两个开发者在开发的时候，一个人做了这个修改，就他们你想你和你的团队成员每个人都对这个代码不爽，你们两个都重构了，那就会进入冲突，就会

进入这个状态。A在这B在那儿。然当你把那个A拉到本地的时候，你就会看到这个冲突状态。你就知道你看冲突现场hello的内容是这样的。

[01:16:57] 然后你可以看到，现在未提交的区域，在未提交的区域，你看在index里面有有好几个，你看hello点C这是你看不到的对吧？这是他还知道Git悄悄的在这里面留了三个版本对吧？叫base，叫ours，irs，然后这里但是他给你看到那个合多点，其实是是这样的一个有圆括号的这个，然后你可以让它resolve。你看那些临时文件没有了。

[01:17:42] 这个merge创建了一个这样的commit object，它的tree是这个，它有两个parent，它有两个parent，然后它有然后你看到这个指向的关系被正确的保留下来了。你看它的tree option选的是这个版本。好，我就不再展示了，所以看到玩coding很好用对吧？玩co很好用。我觉得就你们这么大的时候，这么多年大年纪，尤其是你们有时间的时候，你们最大的乐趣就是外部co的一个这个。然后web code完了以后就各种调，然后跟着他学习，很快你的知识就爆炸式的增长。

12 用 AI 构建可视化编程环境

仙1-107老师

结果

```

...c
int main(int argc, char **argv) {
...}

...
* da07bef (B) merge: 合并分支 A 到 B
(merge commit, 双亲 27e7fd3 + 94d2b56)
|\
| * 94d2b56 (A) refactor: main 改用 void
参数 | 27e7fd3 refactor: main 增加命令行参
数 |
|/

工作区干净。A 已并入 B. 冲突解决完成。

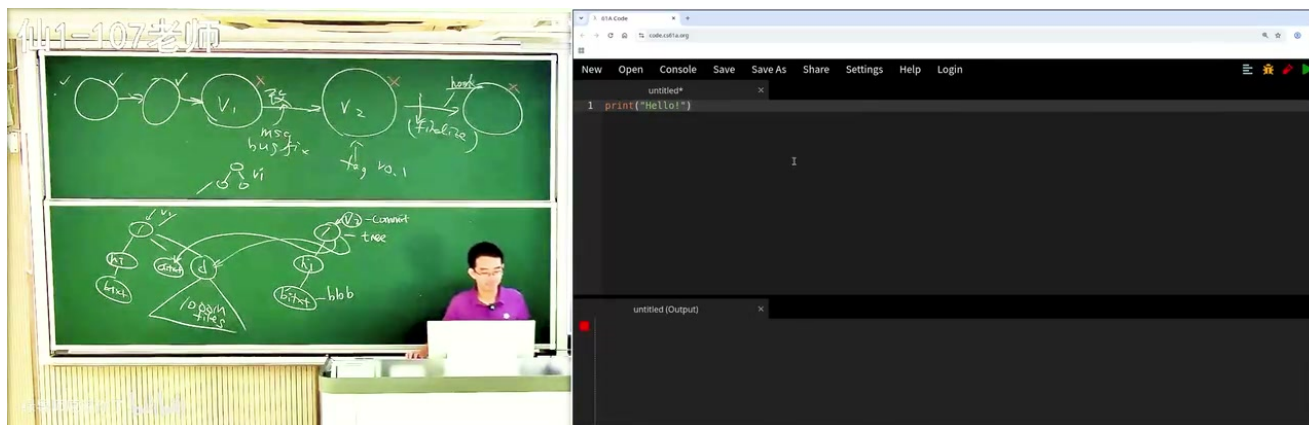
Recap: 解决冲突 = 手工编程去掉
<<<<<<=====>>>>>> 选定内容 + git
add (清除 unmerged stage) + git commit
生成带双亲的 merge commit: git log
--graph 可见合并拓扑。

```

绿导师原速你了

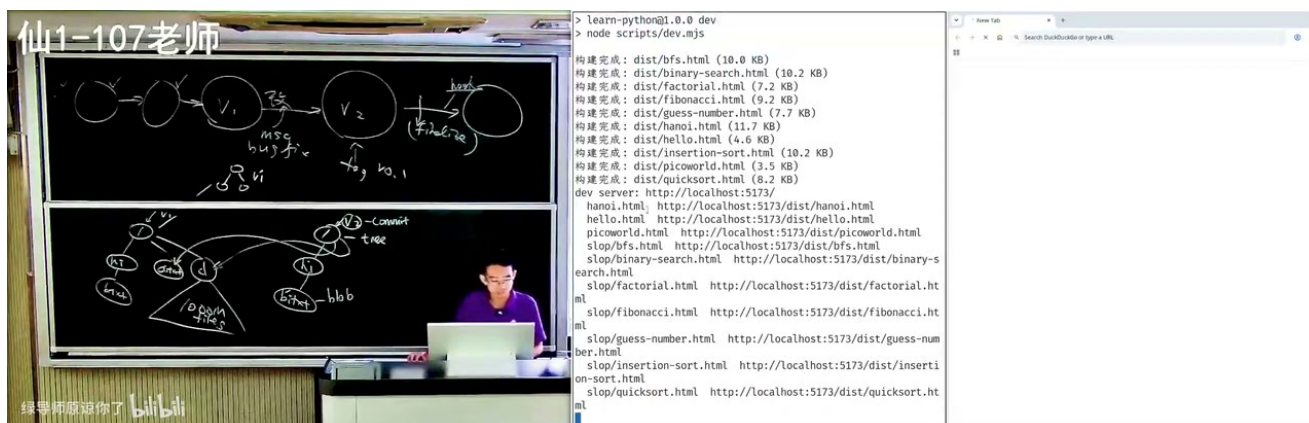
画面 01:18:00

[01:18:30] 然后再举个例子，你们做CS你们上CSC上那个SICP的时候，有没有用过这个官方，这应该是官方的，这个code点C6161A打到你没用过吧。这是一个Python ID我放大一点，这很好用的。你看这是一个如假包换的Python ID你可以运行它。可能他在加载一个Python的runtime库到本地，但是可能他这个库在我这里访问出来并没有很快的地方。但这是就这是一个ID完就基本上完整功能的ID然后他可以吧？你看他CS，ok他需要我老，ok没问题，他可以完成作业。



画面 01:19:00

[01:19:28] 好，最近因为我们有一个小项目，所以我们也被迫外部扣了一个玩意儿。类似的东西，然后你已经看到这里里面有slo其实slo都是就是我直接让AI就是一句话去生成的，它可能也不是一个ID级别的东西。但是但是我觉得效果还是不错的，在架构设计上我稍微做了一点就我我稍微做了一点自己的把控，我把关了。

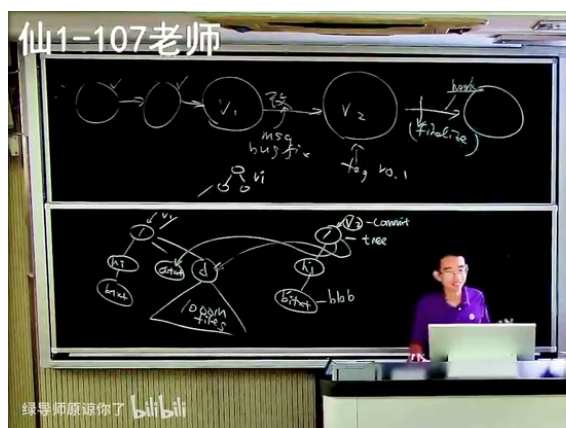


画面 01:20:00

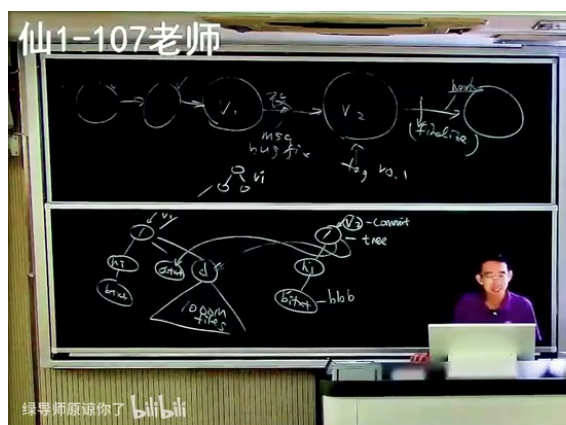
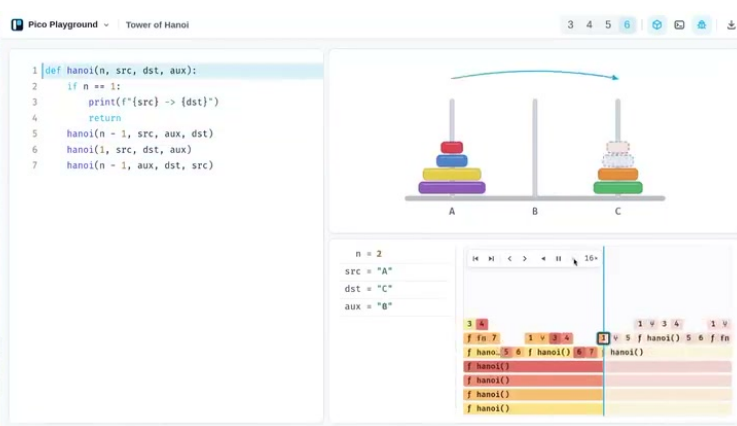
[01:20:09] 所以这个系统的架构还算比较好。它可以它既可以作为一个网页的应用，去教大家编程，也可以作为一个electron的，像vs code一样，一个EX1双击就能打开的。然后它可以可视化任何任何程序。比如说因为我们要给小朋友，给这个三四年的小朋友叫Python，这是一个很有挑战性的事情。因为第一节如果你教他们拍time的话，就全完蛋了，对吧？

[01:20:45] 所以我要要做一个极简的界面，整个都是web部的，就是都是A老，我也不再管控它的代码质量，但我对它的架构其实有把控。我会得到一个什么呢？我第一节要给小朋友讲Python，我就要给他一个印象，Python很好玩对吧？Python很好玩。然后Python很好玩，我就说好，我可以把一个整数变成一个字符串，这个是你们熟悉的。你有没有想到我把一个unique code字符的块块编码成一个整数，然后再把它decode回来，然后还可以hello乘以10，对，我就可以打印十份。Ok这是这是这是上的第一节。

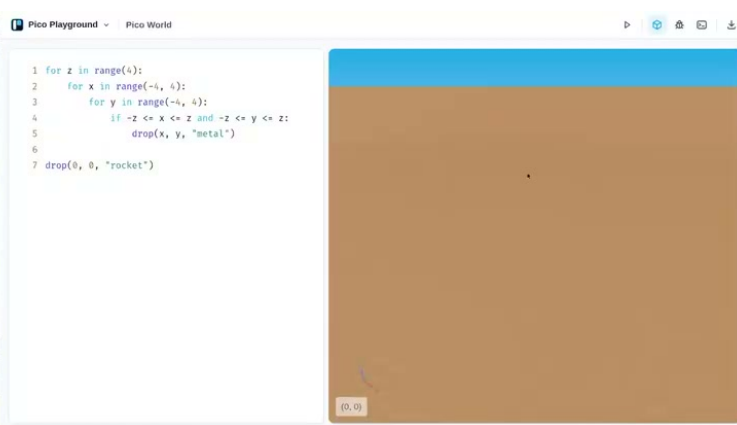
[01:21:29] 然后如果你要讲汉诺塔，我有debugger，这是我的调试器，你每次都可以。你看我在递归之前，我会去告诉你这个它应该怎么移动对吧？然后我可以往后播放，它就会真的移过去？我可以换一个多一点的，然后我可以往前拖，也可以往后拖，对吧？看起来就是一个不错的可视化，这也是外部的这也是完全外部的，上课的负担就轻多了，负担就轻多了，但这还不是我们真正觉得好玩的。



画面 01:22:00

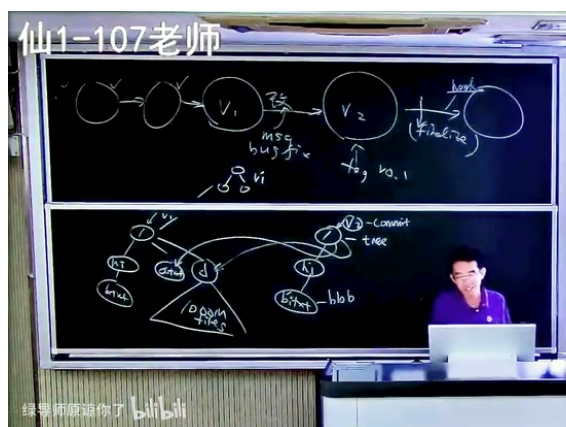


画面 01:22:11

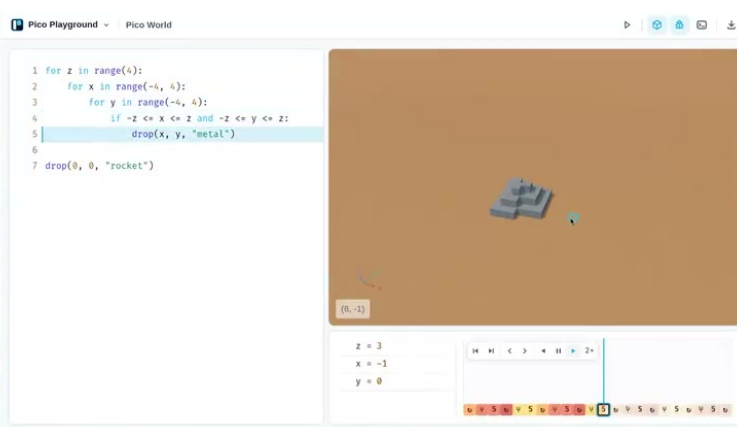


[01:22:14] 你想三四年级的小朋友能编什么程序呢？对吧？三四年的小朋友能编什么程序呢？就这种就这么玩，我给他一个AI给他一个世界，然后他可以可以玩起来，我也可以调试它，对吧？

[01:22:31] 这是一个flat，完全flat的程序。比如说对，你可以看这个程序运行的过程。一个ZXY的循环，然后你可以看到这个方块在从上往下掉，每一次都是真实的，我可以往前走往后退，然视觉效果你显然我就稍微调了一下，所以你看起来就不那么slap了，对吧？看起来包括你看左边这个坐标轴是坐标轴也是正确的。我还骂了一下AI第一版，它实现的坐标轴不符合我的预期，对吧？

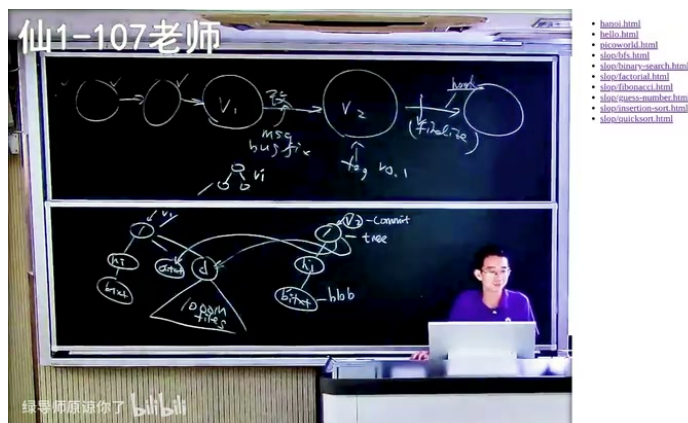


画面 01:23:00



[01:23:16] 最后还可以play，你看这动画是平滑的，我在左右拖的时候它是平滑的，平滑动画。然后你想我们的快乐来自于哪里？你们刚开始学编程的时候就一个二重循环，人打一个星号，打一个三角形你不

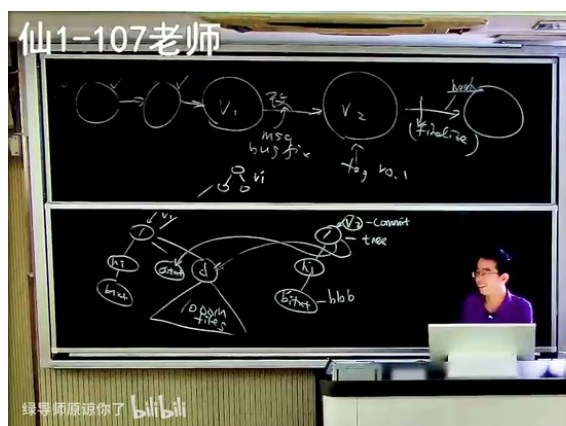
就很开心吗？然后我现在一样买对吧？你学Python，然后你什么都你就可以3D打印了，对吧？这是一个3D打印的程序，再叠别你可以叠，可以挖人，就创造了一整个整个世界，然后剩下所有东西都是直接AI stop。



画面 01:23:46

[01:23:50] 我管都没有管，程序也没有管。比如说number的递归对吧，那是number d归，然后insertion。我回到头上开始播放对吧？插入排序是怎么样一个一个比较的，然后你看到就太容易了当我当然我是有一个skill可以把它编译，就是我有一个编译的skill可以把它编译成把任何一个你想要的东西编译成一个在在模型的能力下，对吧？我还试了这个DC flash division的版本，它就是国产的模型，便宜的模型就可以出出课件。

[01:24:32] 反正我觉得效果已经达到一个完全可用的状态，然后就可以去挣钱了，对吧？在今天你们真正有价值的品质就是你们愿意去花一点时间学习。然后你很快就会发现，在求职的市场上面，把细节做好的这个bar不断的会提高。因为一旦只要有人学会了，招人的单位就会用这个标准来衡量你。



画面 01:25:00

愿意学懂，成了真正有价值的品质

“把细节做好”的 Bar

- 大学里没训练过，真上了战场就都得死
- 很多大学老师真正的好东西打开都没打开过，就只会照本宣科、断章取义
 - 我真的无法接受看公众号学习依然心安理得的老登

好消息：真正学懂和成绩是正相关的

- 愿意学懂，就成了真正有价值的品质。

遗憾的是，自己的学习时间非常有限



[01:25:00] 你想我在面对一个研究生的时候，问他kit merge了以后会发生什么？他没有任何的概念和一个在你们每一学每一个专业课，你学每一个算法，学每一个细节的时候，你都在AI的指导下帮你巩固了这个概念，真正消化的这个概念。你可以从零开始构建这些概念的时候，这两个人他们在这个job market上面的竞争力，当然我觉得可能还有一件事情会发生，就反正AI太强了，所有的人都没有竞争力了。那也许这是就这是最好的就这可能是最好的状态，否则你想想你们的危机感？这个世界上有很多那样的人，然后你们如果不成为那样的人，你上了这个job market你就死了，对吧？

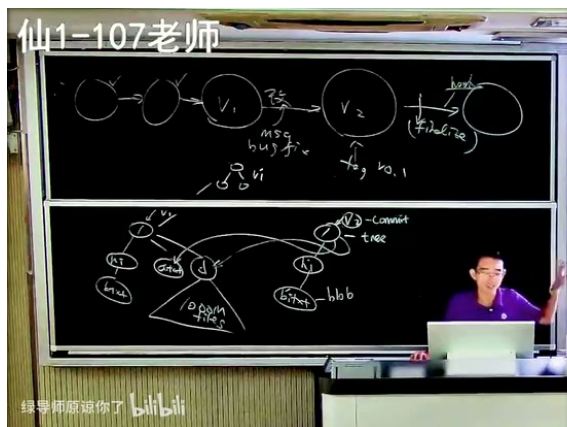
[01:25:44] 所以我觉得上课耽误学习是真实存在的。因为你们的很多老师真正好的东西打开都没有打开过那些真正好代码，真正好的项符合工程规范的项目。他对着那个课本对吧，他就对着那个课本讲一遍。和今天你看到的这个AI直出的东西。就刚才整个的框架，我大概我就是两天的碎片时间就全部完成了。包括视觉效果的调整就完成了，就没有任何的成做这些事情已经没有任何的成本了。对呃所以我我我说我马上就淘汰了，我自己学习的时间是非常有限的。我很想学很多东西，但是我不像你们那么多有那么长时间，大概是这样的。

13 最佳实践、Conventional Commits 与线性历史

[01:26:30] 对，我们讲了一个Git，然后其实所以今天讲的，虽然我好像在讲Git，其实我讲的是best practice。这里面的best practice还有好多，比如说刚才你如果有注意力会注意到AI给了我们一些。其实AI还是挺slow p的。

[01:26:55] 我你看前面的commit message都是英文，都是一行加上一个详细解释。然后你看它就开始自由发挥，他开始混了英文和中文了。但还是还好，他还是遵循了这样的一个，有些人有scope对吧？我干了什么？然后一个总结，再有message body这样的格式。你们看到很多就知道这个东西存在它其上是有标准的。

仙1-107老师



绿导师就逼你了 bilibili

```

Author: Yanyan Jiang <jiangyy@outlook.com>
Date: Tue Sep 8 19:37:05 2026 +0800

merge: 合并分支 A 到 B, 保留 main 的命令行参数签名

commit 27e7fd3af2dc4d1696b74d9f55cb134eacab4627
Author: Yanyan Jiang <jiangyy@outlook.com>
Date: Tue Sep 8 19:34:26 2026 +0800

refactor: main 增加命令行参数

commit 94d2b56c7e4c28aeeef064203141586b52269483 (A)
Author: Yanyan Jiang <jiangyy@outlook.com>
Date: Tue Sep 8 19:34:26 2026 +0800

refactor: main 改用 void 参数

commit 196a2a9cb32d43c41a8da5272c8cf3245ebc7a4c (main)
Author: Yanyan Jiang <jiangyy@outlook.com>
Date: Tue Sep 8 18:58:30 2026 +0800

chore: ignore compiled hello binary

commit 9cc44b0f4fa8412d4fc2c35d13c40638f39b2682
Author: Yanyan Jiang <jiangyy@outlook.com>
Date: Tue Sep 8 18:58:30 2026 +0800

feat: add hello world program

Print nothing yet; scaffold a minimal main().

(END)

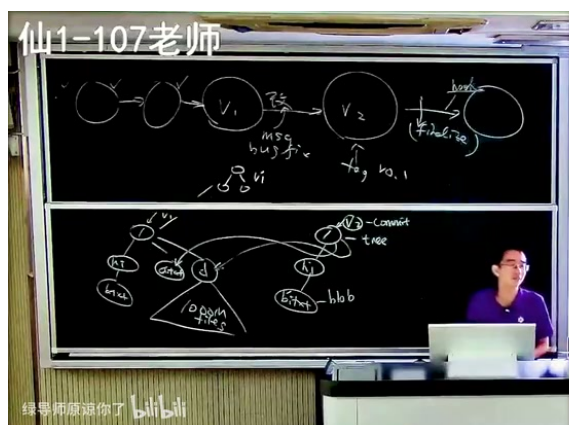
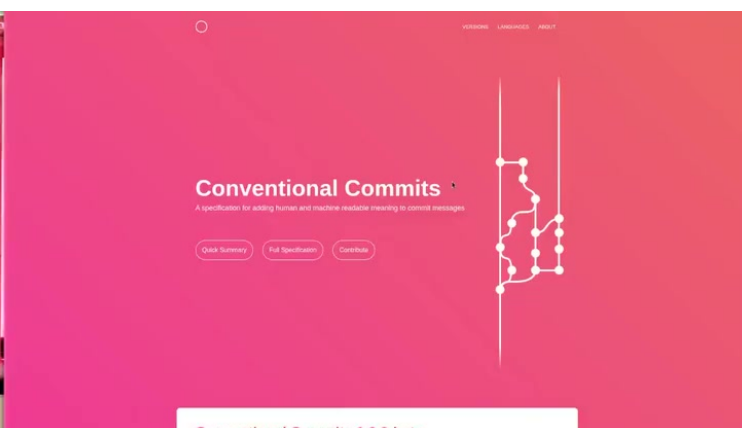
```

画面 01:27:00

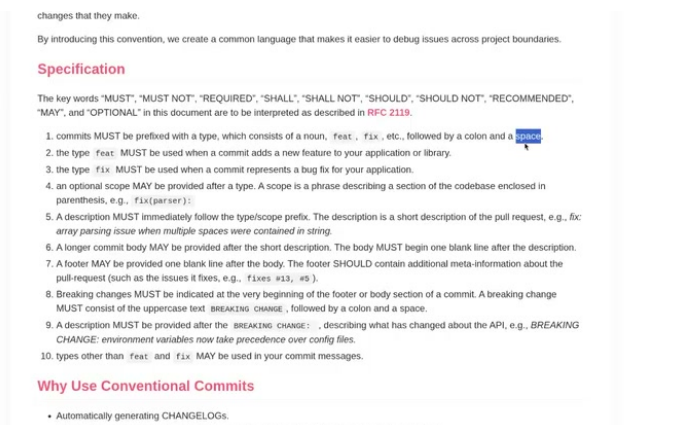
[01:27:26] 在今天你已经可以找到这样的叫conventional commits。Conventional他告诉你这个文档很短？这个specification commits must be prefixed by a visa type是一个none？包括feature，我如果我要引入一个feature，我要fix一个bug？要一个冒号，一定要有一个空格，对，这些都是规范。然后有的时候就比如说我和同学合作的时候，高血压时间还是比较多的。



画面 01:27:28

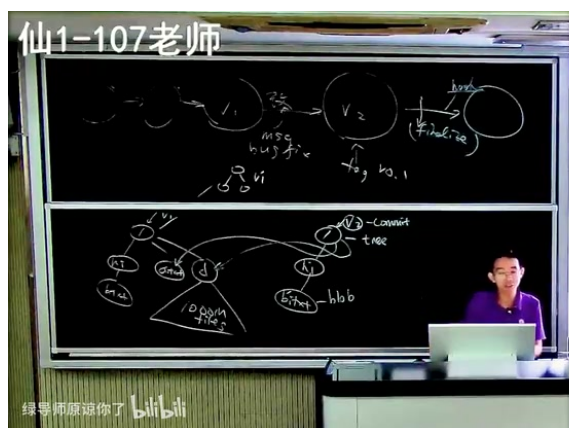


画面 01:28:00



[01:28:00] 比如说如果文档一般来讲，比如mark down的文档，英文和中文之间要有一个空格，对吧？这是我不知道什么时候大家开始用这个可换成，但是好像大家都会用这个程对吧呃对吧比如说你好，这是hello点C对我来说这是肌肉记忆。但如果我看到就是我我最难受的是一个文档里面有一些是这样，你们注意力就是你你们注意到了吗？当我的就我看到这个时候我就是浑身难受。你就你你哪怕这样我都会认为好像你是在遵循某一种对吧？你是保持一致的，但是如果你不一致的话，anyway我真的是有点有点头大。

[01:28:50] 这里有很多，所以AI会把这些细节给做好。而这些工程细节其实都是为了工程师在面对更大项目的时候，它有一种一致性和规范。没有，它是一种比如说你的代码要以同样的方式排版，很自然的你的文档也要以同样的方式排版，这样的一个人维护起来会更好。虽然我觉得在AI时代可能也好像也不重要了，AI都会帮你帮你解决。所以AI我还是很喜欢的，他所有用AI的学生都不会生成那样的，至少对，至少不会形成那样的style。然后kit还有很多的best practice，比如说你可以在网上找到各种commit often perfect leader，publish once? 这里有很多这些其实都是有工程实践的经验。你们回去对回去读一下这些文档，让让AI帮你，你就会学到学到更多，对吧？



画面 01:29:00

更多的 Best Practices

To exit full screen, press and hold [Esc]

比如，你会写 Commit Message 吗？

• Conventional Commits

- `fix(parser): accept repeated spaces between arguments`

- `fix` 类型 + 范围 + 行为变化：人可以浏览，工具可以分类、生成 changelog。

Diff 留下改动，Message 留下理由

- 正文示例：旧解析器把连续空格当成空参数，导致复制来的命令失败；现在忽略多余空格，保留引号内空格。
- `fix`：只能说明“修了 bug”：触发条件、设计理由和 issue 链接，才能帮助后来的人判断“这段代码能不能删”。

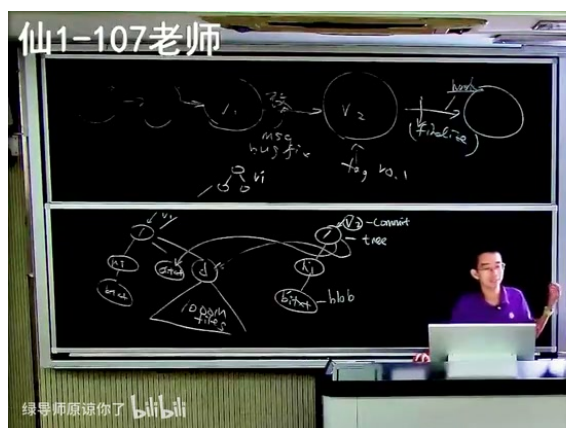
1/18

[01:30:01] 比如说为什么要及时commit？是因为在复杂的项目里面，随着你修改的东西越多，你错的东西就越可能多。你在小项目刚开始的时候，其不要紧急，一次提交个2 3000行代码都ok但是一旦这个项目里面有错综复杂的关系以后，你的一个大的commit可能会break这个项目里面多处的东西，引入多个bug，后面再要引入多个fix来fix它。那这时候你有一个很大的commit？比如我们有一个一万行的commit，然后后面你修了100个bug，你都说这是V2产生的这就很糟糕对吧？甚至修不过来，你也不知道V2修好没有。

[01:30:39] 而如果你每一次的comm都是很小的，都只做一个修改。那冤有头债有主，你这个bug就是你写的好，我修完了也就修完了。这两个是可以配对的，他给了我们审计的能力，也给我们一些。比如说你改了很多，然后不小心真的你的自己的电脑就损坏了，物理损坏了。那你还没有来得及推到远端，你做一个小的提交，立即push，你就可以得到一个互联网上的备份，你的电脑马上坏了也好，对吧？

[01:31:13] 这个AI是很清楚，他知道你可以rereveal，可以revert，可以把二分查找刚才刚才找到的。然后还有一些，我后面可能会讲，比如说hooks和CI然后这样的以及你们如果还有惊人的注意力应该观察过，你们的项目可能托管在GitHub或者glab上。那么你会看到你的AI agent会试图调用一些GH或者glab的命令，然后他会看到一个什么on放的，然后他又回退到古法编程，又开始写Git的命令。

[01:31:56] 是因为GitHub有有完整的这个实在是太popular了，它太好用了。那么我就值得做一个命令行工具，把non Git的部分就Git。管理了好多代码的快照，但是GitHub上还有issue对吧？我今天可以刷一个文本，我还可以对你的代码做comments。然后这些东西都是不随着这个Git的tree做persistence的。么如果我们想要管理它，做一个命令行工具，我可以去fetch一个issue，然后可以找sign给我的一。你要登录，然后找到sign给我一以后，我还可以把这个issue close了。这个命令工具对a来说就很好用对吧？



画面 01:32:00

Git 之外: gh 与 glab

从问题到合入, 把流程接起来

甚至为了管理 non-git 的部分, 还专门有 gh, glab 这些工具。



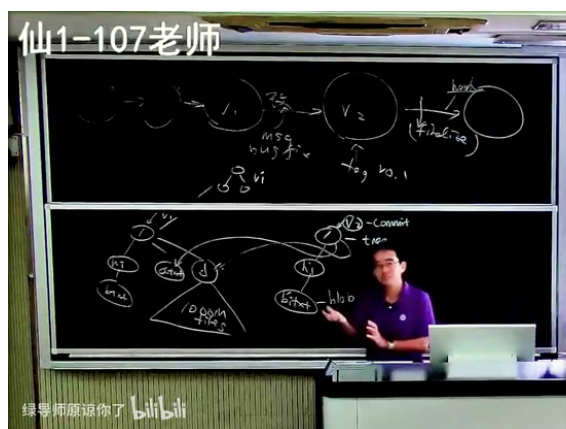
◆ Git 保存和传输版本: gh / glab 操作平台上的 issue、PR/MR 与 CI 状态, 让人和 Agent 都能从命令行完成协作。

[01:32:36] 它可以你装了gh以后, 我就可以在agent里面设置一个go说给我完成一个什么很长, 可以设置一个很长的任务。我会要求他做小步的提交, 做可以审计的提交, 然后怎么分支的规则, 线性的历史或者等等。然后你可以看到我的, 比如说我的这个项目。就大量的这个你看有些对, 从我开始登代码开始, 就大量的这样的提交。然后我很多时候都不管不管他们, 对, 都都不管他们。

14 总结: 用原子变更管理 Agent 项目

[01:33:25] 好, 讲完很多, 其实这门课讲的还是软件工程。所以最后这一小段总结是今天更重要的。这个本质上是一个软件, 它说我们要把项目组织成用atomic change推进的快照, 这是你们今天要记住的事情, 然后所有的一切都是从这句话里面推出来的。

[01:33:51] 你们组织一个软件的项目, 你要管理很多的快照, 这样就可以随时开辟一个新的分支, 做一些尝试。如果失败了, 你就可以把它安全的扔掉。如果成功了, 你就可以把它合并进来, 然后你甚至可以同时推进多个主线, 然后以合并的方式让它完成, 而不会互相干扰。因为如果你们做大作业, 自己古法做大作业肯定有这个感受。你如果要改好几个地方的时候, 改着改着很可能就改乱了。你跟这个部分和那个部分冲突这个部分和那个部分冲突。所以项目管理的本质是人类没有办法处理一团混沌, 很多需求, 很多代码依次做一大步。所以很自然的我们要把需求分解成可以理解, 可以审计的小任务, 一小步一小步来。



画面 01:34:00

回到软件工程

项目 = 由 Atomic Change 推进的快照

可以分支, 可以合并。这是人类成功的经验。

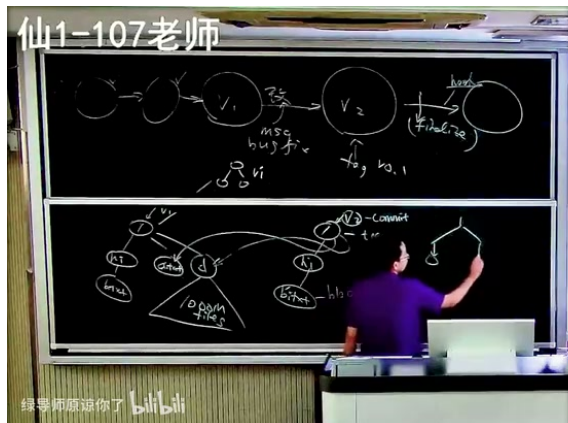
- 人无法处理“一团混沌”
- 必须分解成可以理解、可以审计的小任务
 - 出了问题可以 bisect, 发生冲突可以解决
 - 有些项目要求 linear history: 历史易于追溯, cherry-pick / blame 友好

AI-native 也要避免“一团混沌”

- 在无限大能力模型到来之前, 这条法则依然成立
- ◆ 一个 change 对应一个意图: 出了问题, 才能单独解释、验证、撤销。

[01:34:44] 然后这样出了问题, 你就可以把se可以blame协作的时候, 可以blame发生冲突也容易解决? 两个小任务发生冲突, 甚至比如说我有可能是在这样的一个公共的组先上面, 我可以做到好多事儿, 我

可以做好多。如果每一个都很小，那么如果发生了冲突，可能我能知道哪一个小的提交是冲突的。那么剩下的不冲突的部分，他们合并起来可能会比较容易ok然后包括有些项目会要求linear history，我们做可能更严肃的项目时，也会要求所谓所有的妹。就我们的合并合并可以是合并可以是比如说我这样了，然后我把这个提交合上来。这样就像刚才我们看到一样，我的一个commit可以有两个判人对吧？有两个祖先。然后当然他可能又继续是可能继续又分开又分叉又合并，然后最后会变成一个很典型的Git的结构。



绿导师原班你了 bilibili

画面 01:35:00

回到软件工程

项目 = 由 Atomic Change 推进的快照，

可以分支，可以合并。这是人类成功的经验。

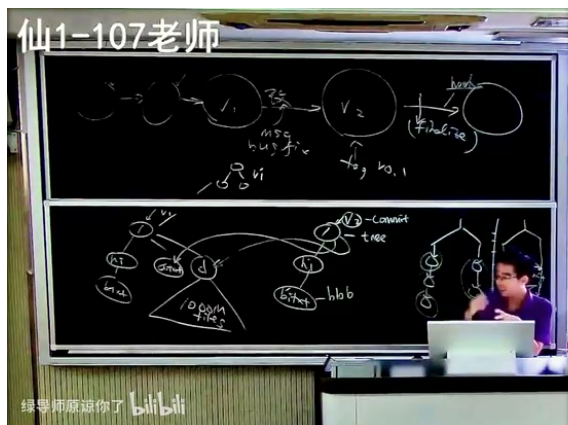
- 人无法处理“一团混沌”
- 必须分解成可以理解、可以审计的小任务
 - 出了问题可以 bisect，发生冲突可以解决
 - 有些项目要求 linear history：历史易于追溯，cherry-pick / blame 友好

AI-native 也要避免“一团混沌”

- 在无限大能力模型到来之前，这条法则依然成立
- ◆ 一个 change 对应一个意图：出了问题，才能单独解释、验证、撤销。

[01:35:47] 但是现在好像大家更流行一个linear history，因为反正都是AI反正它resolve它rebase都很容易。我就强行要求我的整个commit history都是线性的，好像是线性的一样。当然这个也不完全是所有的好处，但至少对于现在我们维护项目来说，他会让历史更易于追溯，而且他对cherry-pick和也也更友好。在无限大能力的模型到来之前，这件事情都是好的。你把Git记录拆分的非常好，你很容易的找到这次修改是由谁引入的。那么AI维护项目的效率也会高，也会省token一些。但有些文件它就可以不用去不用去读了。

[01:36:36] 然后我还有一些经验可以跟大家分享。比如说我们怎么就虽然我们知道是要小步向前进，我们怎么管理一个项目呢？我会在架构设计的阶段，先用单线程，用一个智力比较高的模型跟他讨论。这个其实很重要，因为如果你的软件设计的不好，你后面就是无穷的迭代的。我在第一次课的时候，其实就给大家展示了这个例子。我会把组件库的接口定的非常好，我可以给大家看一下，找一个例子。



绿导师原班你了 bilibili

画面 01:37:00

Agent 能力之内的项目

架构设计：单线程，反复讨论

比如 Pico Playground：和高智力模型讨论，直到收敛。

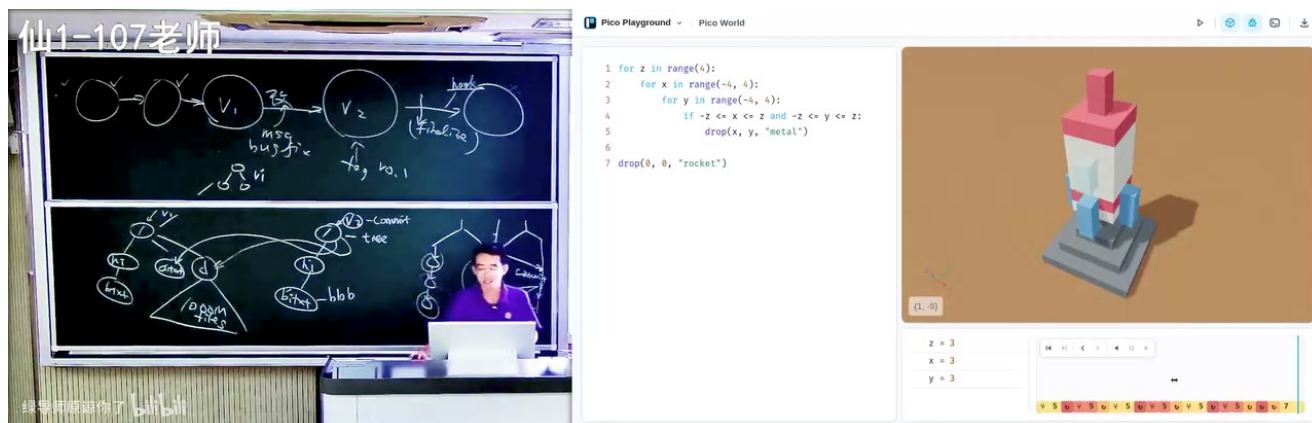
- 将设计落到文档、规则、示例：比如测试要做到什么程度
- 仍然要学操作系统、编译原理、数据库：这里有人类积累的好设计
- ◆ 架构先确定接口与不变量，后续任务才有可以独立行动边界。

实现：有限并发，立即反馈

- $n \leq 3$ 个 terminal
- AGENTS.md 明确协作规则，例如 linear history
- 人脑并发控制：布置不冲突的任务
- 不管 AI 的质量如何，都立即检查、视觉验收、反馈

[01:37:21] 就是我为什么这么信任AI的代码呢？看起来我这里就这个项目里面有很多互相关联的组件对吧？我们的程序会产生会运行产生结果，然后我拖动这个进度条，它不仅关联到我的程序的编辑器，还要关联到这个上面。后但我把绝大部分的接口都设置成了func就是pure function，包括我的trace是程序的一个pure function。然后大概我有一个标准的程序的这个deriva的graph，我说我你要按照这种方式来

展开。然后所有的全局互相影响的状态，我都要注册到一个全局的地方。然后这个全局的状态，比如说我光标的位置，光标的位置可能会怎么样影响这个可视化，全部都包括这个菜单条这个非常干净非常小的状态。所以你看到的一个这样的playground，基本上核心的共享状态也就只有十几行的共享状态。



画面 01:38:00

[01:38:34] 然后一个十几行的组装，然后剩下所有东西都是黑盒子，我都要求他，我就直接会要求agent在设计这些visualization的时候，禁止看任何库函数的实现，只看接口。所以它的非常其实非常省上的，就几个token就可以做一个做一个这样的做这样的推广。然后一旦这样的架构设计完成了以后，我就会开三个或者开几个terminal。然后我要求他我的agent m有一个明确的协作规则，我看还在不在，可能改过一些。对，没准对我有一些写作规则。

Agent 能力之内的项目

架构设计：单线程，反复讨论

比如 Pico Playground：和高智力模型讨论，直到收敛。

- 将设计落到文档、规则、示例：比如测试要做到什么程度
- 仍然要学操作系统、编译原理、数据库：这里有人类积累的好设计
- ♦ 架构先确定接口与不变量，后续任务才有可以独立行动的边界。

实现：有限并发，立即反馈

- $n \leq 3$ 个 terminal
- AGENTS.md 明确协作规则，例如 linear history
- 人脑并发控制：布置不冲突的任务
- 不管 AI 的质量如何，都立即检查、视觉验收、反馈

画面 01:38:54

```

Rewrite the compile skill and trim the docs
commit 7e42fedbad9b250be2bd429cf30301f7c5945fa8
Author: Yanyan Jiang <jiangyy@outlook.com>
$ npm run dev

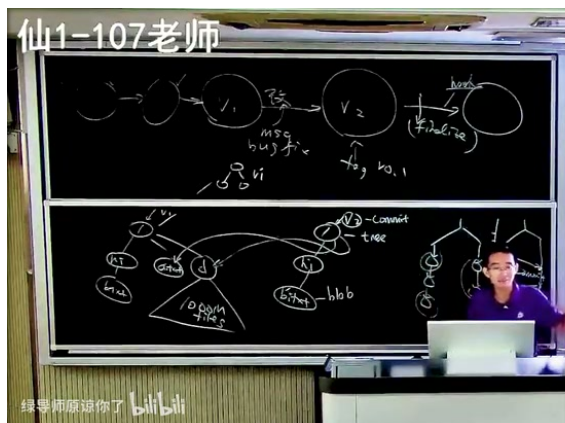
> learn-python@1.0.0 dev
> node scripts/dev.mjs

构建完成: dist/bfs.html (10.0 KB)
构建完成: dist/binary-search.html (10.2 KB)
构建完成: dist/factorial.html (7.2 KB)
构建完成: dist/fibonacci.html (9.2 KB)
构建完成: dist/guess-number.html (7.7 KB)
构建完成: dist/hanoi.html (11.7 KB)
构建完成: dist/hello.html (4.6 KB)
构建完成: dist/insertion-sort.html (10.2 KB)
构建完成: dist/picoworld.html (3.5 KB)
构建完成: dist/quicksort.html (8.2 KB)
dev server: http://localhost:5173/
hanoi.html http://localhost:5173/dist/hanoi.html
hello.html http://localhost:5173/dist/hello.html
picoworld.html http://localhost:5173/dist/picoworld.html
slop/bfs.html http://localhost:5173/dist/bfs.html
slop/binary-search.html http://localhost:5173/dist/binary-search.html
slop/factorial.html http://localhost:5173/dist/factorial.html
slop/fibonacci.html http://localhost:5173/dist/fibonacci.html
slop/guess-number.html http://localhost:5173/dist/guess-number.html
slop/insertion-sort.html http://localhost:5173/dist/insertion-sort.html
slop/quicksort.html http://localhost:5173/dist/quicksort.html

```

画面 01:39:10

[01:39:15] 比如说我让他不需要调用视觉模型去检查一阵。很喜欢做这件事，然后花很多的很多token，然后我也不调用浏览器任务，我在边上放一个另外一块屏上放一个浏览器，然后它是及时刷新的，然后我就开始开三个agent，哪里不满意就骂他。然后你就可以肉眼看着右边那个东西不停的变好。最后问题就问题就解决了，这是一个agent能力之内的项目。然后我非常确信的是会agent能力会不断的增长，然后越来越多的项目能够被agent以这样的方式就是一个比较轻量的讨论，然后剩下完成。这是个非常可怕的事情。就是如果agent快了1000倍，我们还怎么样管理这个版本控制？我们可以下一节课再来讨论。好，今天下课。



画面 01:40:00

当 Agent 快了 1000 倍

一个思想实验：代码产能暴涨

- Agent 可以 1000X 更快地生产代码
- Git“为人设计”的使用方式，可能跟不上这个速度
 - 生成一分钟，人工验收一小时：积压的是等待判断的变更。
 - 即使修改不同文件，也可能同时改变同一个接口约定。

从零设计一个 Agent VCS

- 快照、变更，以及一个 change algebra
- 系统要回答：哪些变更能组合、哪些必须排队、哪些需要重新验证？
- 像数据库处理事务一样，让 Agent 提交“基于某个版本的修改”，由系统控制它何时生效。